

BACHELOR THESIS

Dilshod Khudoyberdiev

Design and Implementation of an AI Agent for Text-to-SQL Conversion in Relational Databases

FACULTY OF ENGINEERING AND COMPUTER SCIENCE

Department of Information and Electrical Engineering

Fakultät Technik und Informatik

Department Informations- und Elektrotechnik

Dilshod Khudoyberdiev

Design and Implementation of an AI Agent for Text-to-SQL Conversion in Relational Databases

Bachelor Thesis based on the examination and study regulations for
the Bachelor of Engineering degree programme

Bachelor of Science Information Engineering

at the Department of Information and Electrical Engineering of
the Faculty of Engineering and Computer Science
of the University of Applied Sciences Hamburg

Supervising examiner: Prof. Dr. Björn Gottfried

Second examiner: Hans Schrader

Day of delivery: April 16, 2026

Dilshod Khudoyberdiev

Title of Thesis

Design and Implementation of an AI Agent for Text-to-SQL Conversion in Relational Databases

Keywords

Text-to-SQL, Large Language Models, AI Agent, Retrieval-Augmented Generation, Streamlit

Abstract

Non-technical staff often depend on data specialists for simple database questions, because writing SQL and understanding database schemas requires experience. This thesis presents a self-hosted text-to-SQL application that allows users to ask questions in plain text and receive executable SQL queries for relational databases. The system follows an agent-like workflow: it retrieves schema information and selected examples to guide query generation, produces a short explanation for transparency, and validates each statement before execution. To reduce risks in business environments, the application enforces strict read-only execution and blocks unsafe statements. A lightweight interface also allows users to manage training examples, so the system can be improved iteratively for domain-specific terminology. The prototype is implemented as a Streamlit web application and evaluated using a synthetic but realistic dataset. The evaluation focuses on usability for non-technical users, reliability of generated SQL, and the effect of training data updates on query quality.

Dilshod Khudoyberdiev

Thema der Arbeit

Entwurf und Implementierung eines KI-Agenten zur Text-zu-SQL-Umwandlung in relationalen Datenbanken

Stichworte

Text-zu-SQL, Große Sprachmodelle, KI-Agent, Retrieval-Augmented Generation, Streamlit

Kurzzusammenfassung

Nicht-technische Mitarbeitende sind bei einfachen Datenbankfragen oft auf Spezialisten angewiesen, weil SQL und Datenbankschemata schwer zu verstehen sind. Diese Arbeit entwickelt eine selbst gehostete Text-zu-SQL-Anwendung, mit der Nutzer Fragen als Text eingeben und ausführbare SQL-Abfragen für relationale Datenbanken erhalten können. Das System arbeitet wie ein Agent: Es nutzt Schema-Informationen und passende Beispiele, erstellt daraus eine SQL-Abfrage, liefert eine kurze Erklärung und prüft die Abfrage vor der Ausführung. Um Risiken im praktischen Einsatz zu vermeiden, werden nur reine Leseabfragen erlaubt und unsichere Anweisungen konsequent blockiert. Zusätzlich gibt es eine einfache Oberfläche, mit der Trainingsbeispiele verwaltet werden können, um das System schrittweise an domänenspezifische Begriffe anzupassen. Der Prototyp wird als Streamlit-Webanwendung umgesetzt und mit einem synthetischen, aber realitätsnahen Datensatz bewertet. Dabei stehen Bedienbarkeit, Zuverlässigkeit der SQL-Generierung und der Einfluss von Trainingsupdates im Fokus.

Contents

1. Introduction.....	1
1.1 Purpose of this chapter.....	1
1.2 Background and motivation.....	1
1.3 Problem statement.....	2
1.4 Industrial context.....	3
1.5 Objectives.....	3
1.6 Thesis structure.....	4
2.Theoretical Background.....	5
2.1 Purpose of this chapter.....	5
2.2 Overview of Text-to-SQL Systems.....	5
2.3 Components and Conversion Process.....	6
2.4 Key Concepts and Terminology.....	8
Relational database.....	8
SQL (Structured Query Language).....	9
Pre-trained language model (PLM).....	9
Prompting.....	9
In-context learning.....	10
Fine-tuning.....	10
Retrieval-augmented generation (RAG).....	10
Schema linking.....	10
Evaluation: exact match vs. execution accuracy.....	10
Minimum Viable Product (MVP).....	11
Data Definition Language (DDL).....	11
2.4.1 SQL basics for Text-to-SQL (syntax vs. semantics).....	11
2.5 Types of Approaches.....	13
2.5.1 Rule-based and template-based systems.....	13
2.5.2 Trained ML systems (PLM-based).....	14
2.5.3 LLM-based systems.....	15
2.6 Applications in Business Intelligence.....	16
2.7 Evaluation Procedure and Project-Specific Test Sets.....	17
2.7.1 Main in-domain test set.....	17
2.7.2 Additional held-out, safety and semantic-error test sets.....	18
2.7.3 How mxQueryChat is evaluated on the project test sets.....	19
Chapter 3 Requirements.....	20
3.1 Purpose of this chapter.....	20
3.2 Stakeholders.....	20
3.2.1 Primary stakeholders.....	20
3.2.2 Secondary stakeholders.....	21
3.3 Functional requirements.....	21
3.3.1 User interaction and visualization requirements.....	21
3.3.2 Data management and system capabilities requirements.....	23
3.4 Non-functional requirements.....	24
3.5 User requirements.....	25
3.6 Use cases.....	26
3.6.1 Use Case 1: Ask a question and get results.....	27

3.6.2 Use Case 2: Correct the generated SQL before execution.....	27
3.6.3 Use Case 3: Request an explanation of the SQL.....	27
3.6.4 Use Case 4: Manage training examples.....	28
3.6.5 Use Case 5: Trigger training updates from the UI.....	28
3.6.6 Use Case 6: Handle unsafe or off-topic input safely.....	28
3.7 MoSCoW prioritization.....	29
Chapter 4 Design.....	31
4.1 Purpose of this chapter.....	31
4.2 Design goals and scope constraints.....	31
4.3 High-level architecture.....	31
4.4 User interface design.....	32
4.5 Core interaction flows.....	34
4.5.1 New Question flow.....	34
4.5.2 Training Data flow.....	35
4.6 Prompting, retrieval, and grounding behavior.....	35
4.7 Safety design.....	36
Chapter 5 Implementation.....	37
5.1 Purpose of this chapter.....	37
5.2 Implementation stack and rationale.....	37
5.3 Code structure and module responsibilities.....	38
5.4 Dataset and schema handling.....	39
5.5 Training examples and training update implementation.....	40
5.6 SQL generation, review, and execution pipeline.....	40
5.7 Optional SQL explanation.....	40
5.8 Guardrails and failure categories.....	40
5.9 Result display and chart rules.....	41
Chapter 6 Evaluation.....	42
6.1 Purpose of this chapter.....	42
6.2 Research questions.....	42
6.3 Experimental setup and reproducibility.....	42
6.4 Test sets.....	43
6.5 Metrics.....	45
6.6 Results reporting plan.....	46
6.7 Error analysis.....	48
6.8 Training loop effect (before/after).....	49
6.9 Usability observations.....	50
Chapter 7 Discussion and Limitations.....	52
7.1 Purpose of this chapter.....	52
7.2 Interpretation of the main findings.....	52
7.3 Threats to validity.....	53
7.4 Limitations of the MVP.....	54
7.5 Future work.....	55
References.....	56
Appendix.....	57
Declaration.....	65

List of Figures

Figure 1: High-level workflow of the mxQueryChat Text-to-SQL system 2

Figure 2: Text-to-SQL pipeline illustrating the main stages from a natural language question to SQL execution 8

Figure 3: UML Use Case Diagram of mxQueryChat 29

Figure 4: New Question View Layout 33

Figure 5: Training Data View Layout 33

Figure 6: Sequence Diagram - New Question Flow 34

Figure 7: Sequence Diagram - Training Data Update Flow 35

Figure 8: High-level architecture of the system 38

Figure 9: Conceptual ER Diagram of the DuckDB Dataset 39

Figure 10: Failure Category Distribution 48

Figure 11: Before/After Comparison of Training Impact 50

List of Tables

Table 3.1: Primary stakeholders	20
Table 3.2: Secondary stakeholders	21
Table 3.3: User interaction and visualization requirements	22
Table 3.4: Data management and system capabilities requirements	23
Table 3.5: Non-functional requirements	24
Table 3.6: User requirements	26
Table 3.7: MoSCoW prioritization	29
Table 6.1: Evaluation Environment	43
Table 6.2: Domain Test Set Overview	44
Table 6.3: Local Held-Out Benchmark Set	44
Table 6.4: Safety and Robustness Test Cases	45
Table 6.5: Domain Results Summary	46
Table 6.6: Local Held-Out Benchmark Results	47
Table 6.7: Safety Results Summary	48
Table 6.8: Semantic Error Examples	49
Table 6.9: Training Update Impact	50
Table 6.10: Telemetry-Based Usability Signals	51

List of Abbreviations

AI Artificial Intelligence

BI Business Intelligence

DB Database

DDL Data Definition Language

DML Data Manipulation Language

ERD Entity Relationship Diagram

LLM Large Language Model

PLM Pre-trained Language Model

NL Natural Language

NLIDB Natural Language Interface to Databases

RAG Retrieval-Augmented Generation

SQL Structured Query Language

UI User Interface

UML Unified Modeling Language

MVP Minimum Viable Product

1. Introduction

1.1 Purpose of this chapter

This chapter introduces the topic and motivation of the thesis. It explains why natural language access to relational databases is relevant, what practical problem the thesis addresses, and what role Text-to-SQL systems can play in this context. In addition, the chapter defines the research objective, outlines the scope of the work, and gives a short overview of the structure of the thesis.

1.2 Background and motivation

Many organizations store operational and customer data in relational databases. A relational database stores data in tables (rows and columns). In practice, people access this data using SQL (Structured Query Language), which allows selecting, filtering, grouping, and joining data across tables.

SQL is powerful, but it creates a barrier for many employees. Non-technical staff can often describe what they need in plain language, but they do not know the database schema and cannot reliably write SQL. As a result, they depend on analysts even for routine questions (e.g., “Which federal states had the highest ticket revenue last quarter?”). This slows down decision-making and creates repetitive work for data teams.

Text-to-SQL systems aim to reduce this gap by converting natural language questions into executable SQL queries. Recent overview papers emphasize that generating syntactically valid SQL is not sufficient: the query must also match the user’s intent and the database structure, otherwise it may execute but return a wrong answer[1, 2, 3, 4].

In recent years, Large Language Models (LLMs) have become a common component for Text-to-SQL. In this context, an LLM can generate SQL from a question when it is given enough schema-related context and examples[1, 2, 3]. However, missing or incomplete context can lead to hallucinations (invented tables/columns) or semantically wrong queries (e.g., wrong joins or filters). A common strategy to reduce guessing is Retrieval-Augmented Generation (RAG), where the system retrieves relevant schema elements and similar examples before generation and includes them in the model input[1, 3].

Based on these ideas, this thesis develops **mxQueryChat**, a self-hosted Text-to-SQL application that translates natural language questions into SQL queries for relational databases and executes them in a controlled read-only environment. For brevity, mxQueryChat is referred to as “**the system**” in the remainder of this thesis. **Figure 1** provides a high-level overview of the workflow used in this thesis, highlighting retrieval-augmented context and read-only validation as key design elements.

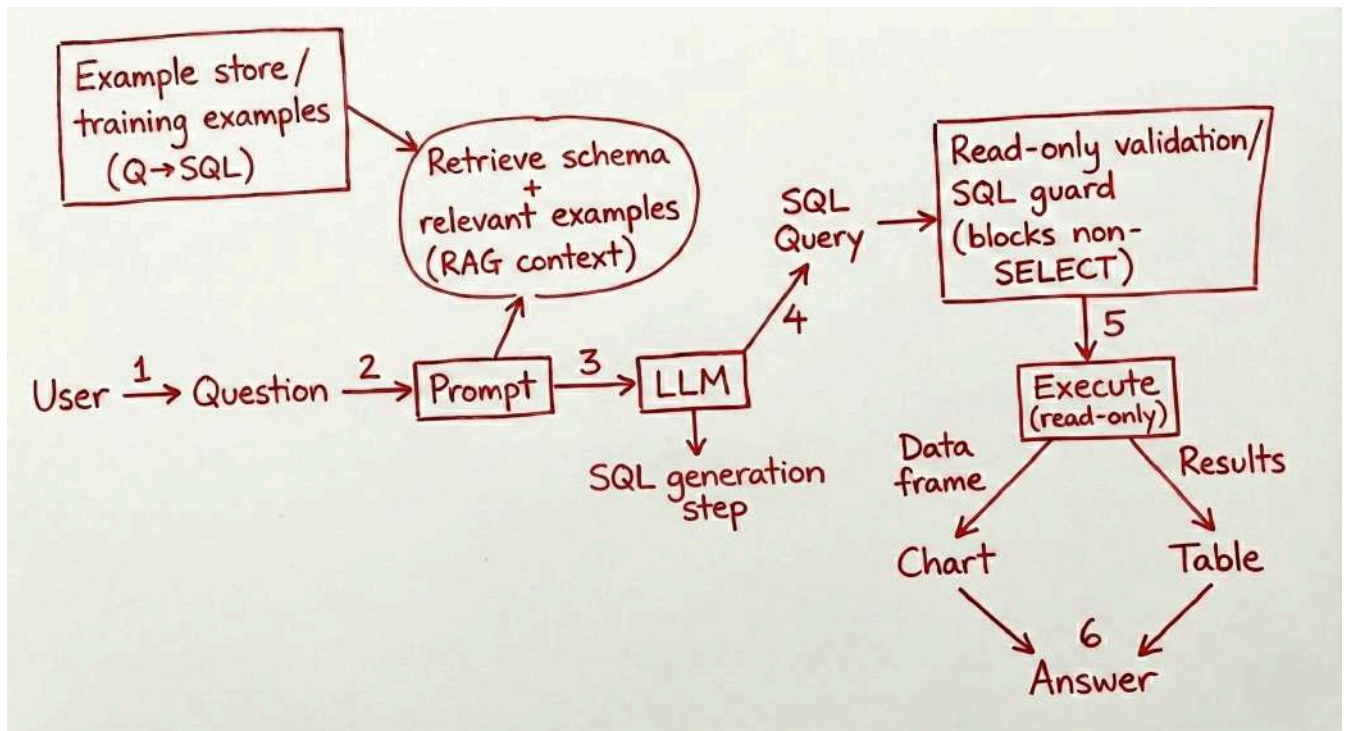


Figure 1: High-level workflow of the mxQueryChat Text-to-SQL system

1.3 Problem statement

Even with LLMs, deploying Text-to-SQL in a company remains difficult. The main issues are:

1. Schema understanding

A database has a **schema**, meaning the database structure: tables, columns, data types and relationships. The system must use the schema correctly. If it selects the wrong column or the wrong join path, the result can be wrong even if the SQL runs.

2. Schema linking

Schema linking means mapping the words in the question to the correct tables and columns. For example, “revenue” might mean *price * quantity* or it might be a stored column like *total_amount*. This mapping is a common source of errors in Text-to-SQL according to multiple literature reviews[1, 2, 4].

3. **Correctness beyond “looks right”**

A query can be syntactically correct but semantically wrong (wrong join, wrong filter, wrong aggregation). Recent Text-to-SQL methods propose decomposition and verification steps to reduce such errors. This suggests that single-step generation is often not enough for complex questions[5, 6].

4. **Grounding in database values**

Many users refer to **concrete values** that exist in the database, such as product names, customer segments, city names, or internal codes. In many cases, the user does not know the exact spelling or the exact identifier stored in the table. If the system “guesses” a value (or invents one that does not exist), the resulting SQL may still execute but it can return incomplete or misleading results. For this reason, **value grounding**, linking the user’s words to actual values present in the database, is a major challenge for Text-to-SQL in realistic business settings [7].

5. **Robustness under real usage and schema change**

Real deployments introduce additional problems: user questions vary a lot, schemas evolve over time and small modelling choices can change results. Work based on real user queries studies robustness and shows that system design choices matter when moving from demos to deployment[8]. Other recent work explicitly evaluates robustness against schema evolution and shows that schema changes can break Text-to-SQL behaviour[10].

6. **Safety and trust in business settings**

In companies, running generated SQL must be safe. A system should block any statement that modifies data or changes the schema. It should also show the SQL so users and admins can verify what will be executed. This is especially important when the user is not technical.

1.4 Industrial context

This thesis targets a **privacy-focused company environment**. In this context, *self-hosted* means the system runs inside internal infrastructure and does not send schema information or query context to external cloud APIs. This constraint matters because database schemas and business data can be sensitive. A local deployment supports compliance requirements and reduces unnecessary data exposure.

1.5 Objectives

Based on the motivation in Chapter 1.2 and the challenges in Chapter 1.3, this thesis designs and implements mxQueryChat, a self-hosted Text-to-SQL system for relational databases that helps non-technical users query data in natural language while keeping execution safe and SQL transparent. The system uses an *agent-like workflow*, meaning it retrieves relevant context, generates SQL, validates it, and

only then executes and explains the result, so checks can be applied before anything runs.

This thesis has the following **objectives**:

- **O1 Grounded SQL generation:** Use schema-aware prompting with retrieved context (RAG) so SQL generation stays grounded in the real schema and relevant examples.
- **O2 Reliability improvements:** Improve correctness using decomposition and/or verification steps suggested by recent Text-to-SQL methods.
- **O3 Safety:** Enforce strict read-only execution and block unsafe SQL patterns to reduce operational risk.
- **O4 Domain adaptation:** Support user-managed training examples so the system adapts to domain terms and recurring query patterns over time.
- **O5 Evaluation:** Measure performance using execution success, error categories, safety blocking rate, response time, and improvement after adding examples, following robustness-oriented evaluation practices.

1.6 Thesis structure

The thesis is organized as follows:

- **Chapter 1 (Introduction):** motivates the problem, summarizes the industrial context and practical challenges, and states the objectives.
- **Chapter 2 (Theoretical Background):** reviews the main concepts and approaches relevant to this thesis, including Text-to-SQL methods, schema linking, retrieval-augmented generation, verification strategies and evaluation practices.
- **Chapter 3 (Requirements):** derives stakeholders, use cases, and functional and non-functional requirements, and prioritizes them using the MoSCoW method.
- **Chapter 4 (Design):** describes the architecture and interaction flow of mxQueryChat, including context retrieval, SQL generation, validation, and user interface decisions.
- **Chapter 5 (Implementation):** details the technical implementation, module structure, and error handling.
- **Chapter 6 (Evaluation):** reports test results and evaluation outcomes, including the effect of training examples on reliability and observed failure modes.
- **Chapter 7 (Discussion and Limitations):** summarizes findings, discusses limitations, and outlines future work.

2.Theoretical Background

2.1 Purpose of this chapter

This chapter provides the theoretical background needed for the thesis. It explains the main concepts on which the work is based, including natural language interfaces to databases, Text-to-SQL systems, schema linking, retrieval-augmented generation, large language models, and supporting software concepts such as the Minimum Viable Product (MVP). The purpose of this chapter is to establish the technical foundation for the later analysis, design, and implementation chapters.

2.2 Overview of Text-to-SQL Systems

Text-to-SQL is the task of turning a user's question written in normal language into an SQL query that can be executed on a relational database. A relational database stores data in tables with rows and columns. In research, Text-to-SQL is often described as a *natural language interface to databases (NLIDB)*. This means: instead of writing SQL, users can ask questions in everyday language, and the system generates the SQL for them[1, 2].

This topic is important for this thesis because mxQueryChat is designed for business users who want to access company data but do not have advanced SQL writing skills. The system should help them obtain answers without requiring them to learn database details.

However, Text-to-SQL is not only about translating words. The main difficulty is that the system must connect the user's words to the correct parts of the database [1, 2, 3, 6]. This includes:

- selecting the right tables (where the needed information is stored),
- selecting the right columns (the exact fields),
- and using the right relationships between tables (how tables are linked).

This is difficult because business words are often **not stored in one obvious place** in the database. The same word can refer to different columns, or it can be something that must be **computed**. For example, "revenue" might exist as a single column in one database, but in another database it might need to be calculated, such as **price × quantity**, and possibly also adjusted by discounts or refunds.

A key problem is the difference between *syntax* and *semantics*:

- **Syntax** means: "Is the SQL written correctly?" If the syntax is correct, the database can execute the

query.

- **Semantics** means: “Does the SQL actually answer the user’s question?” The query might run successfully but still answer the wrong question.

For example, a system might join two tables using the wrong ID column. The database will still return results, and the output may look reasonable. But the numbers may be wrong because the query connected the wrong rows. This is dangerous in business contexts because wrong results may not be noticed [2,3, 9, 10].

Because of this, many Text-to-SQL approaches emphasize **grounding** [1, 2]. In simple words, grounding means the model should stay connected to the real database schema (real table names, real column names, and real relationships). The goal is that the generated SQL matches both the database structure and the meaning of the user question. In this thesis, this motivation supports design choices in mxQueryChat such as using schema-aware context, showing the generated SQL to the user, and checking the query before execution [1, 2, 3, 8].

2.3 Components and Conversion Process

Most Text-to-SQL systems can be described as a pipeline. **The input** is a natural language question. **The output** is an SQL query that is executed on a database to produce a result. Even if modern systems use large language models, the overall process can still be explained in a few clear steps.

To make the steps easy to understand, this section uses one running example question:

Example question: “Show the top 5 tariff organizations by ticket revenue for 2025.”

A typical pipeline consists of **four** main components.

1) Understanding the user question

First, the system must understand what the user is asking for in *plain terms*. In the example, the question contains several “hidden instructions”:

- **Measure:** “revenue” (what should be calculated)
- **Grouping:** “tariff organizations” (what should be compared)
- **Time filter:** “for 2025” (which period)
- **Ranking and limit:** “top 5” (how results should be ordered and restricted)

A common source of mistakes is ambiguity. For instance, “revenue” might mean *gross revenue* or *net revenue*, and “for 2025” might refer to the sales date or to a reporting period column. If the system interprets

these incorrectly, it may still produce valid SQL but answer a different question [1, 2, 3].

2) Understanding the database schema (knowing what data exists and where)

Second, the system must understand the database structure (*the schema*). A schema is the set of tables and columns, and how tables are connected through keys (relationships). In the example, the system must find:

- Which table contains sales or ticket transactions,
- Which table contains tariff organizations,
- Which columns represent revenue-related values,
- And how to join these tables correctly.

This step matters because relational databases usually split information across multiple tables. If the system chooses the wrong table or a wrong join path, it can produce results that look plausible but are wrong [1, 2, 3, 6].

3) Generating the SQL query (turning the user intent into SQL statements)

Third, the system generates SQL that matches both:

- the **intended meaning** of the question (semantics), and
- the **formal SQL rules** (syntax).

This is where the syntax-semantics distinction becomes important. A query can be syntactically correct (the database can execute it) but semantically wrong (it does not reflect the intended meaning) [1, 2, 3, 6]. For example:

- A wrong join key may duplicate rows and inflate revenue.
- A missing filter for 2025 may compute revenue over all years.
- A wrong aggregation may compute an average instead of a sum.

Many modern systems use large language models (LLMs) to write SQL. Some systems do this in **one shot**: meaning, the model reads the question and immediately outputs the final SQL. Other systems use **several smaller steps**. For example, the model first writes a draft query, then it checks the draft against the database schema and the user's intent, and finally it produces a corrected final query. This *multi-step approach* is used because it makes it easier to catch typical meaning errors (like wrong joins, missing filters, or wrong aggregations) before the query is executed [7].

4) Validating and executing SQL (making sure it is safe and reasonable)

Finally, before execution, practical systems check whether the generated SQL is acceptable. There are two types of checks that matter in business settings:

- **Safety checks:** ensure the query cannot modify data (for example, block INSERT/UPDATE/DELETE or schema-changing statements).
- **Reasonableness checks:** basic validation that reduces obvious semantic errors (for example, checking that referenced tables/columns exist, or that the query is a single SELECT) [1, 2, 8].

After validation, the system executes the query and returns results.

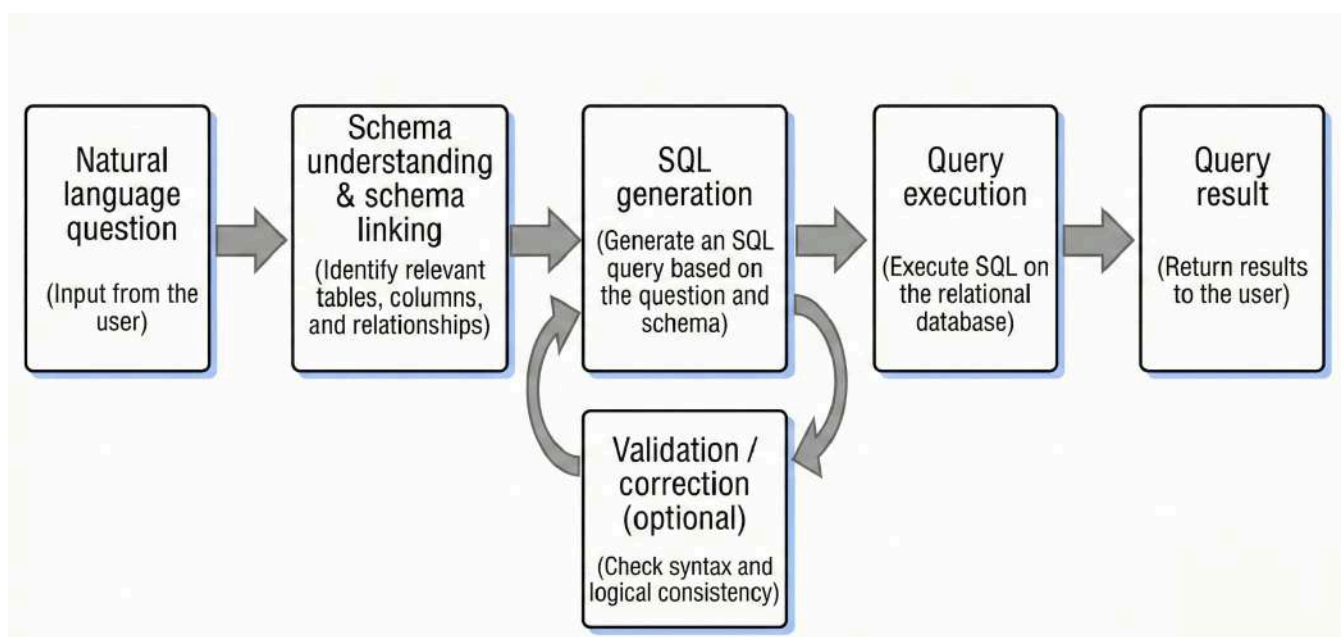


Figure 2: Text-to-SQL pipeline illustrating the main stages from a natural language question to SQL execution and result retrieval

2.4 Key Concepts and Terminology

This section explains the key terms used in Chapter 2 and later design and evaluation chapters. The goal is not to give textbook definitions, but to explain each term in a way that helps a reader understand how mxQueryChat works and where mistakes can happen.

Relational database

A relational database stores data in **tables**. Tables are connected through **keys**, for example a *customer_id* in a sales table that points to a row in a customer table. **Why it matters for this thesis:** mxQueryChat generates queries for relational databases, so it must choose correct tables and correct join paths. A wrong

join can change results even if the query runs.

SQL (Structured Query Language)

SQL is the standard language used to query relational databases. SQL has a strict structure (syntax) and each part has a meaning (semantics). Typical building blocks are SELECT (what to show), FROM (which table), WHERE (filter), GROUP BY (aggregation), and JOIN (combine tables) [1, 2, 3].

Model

A *model* is a system that takes an input and produces an output based on patterns it has learned or rules it follows. **Real-life analogy:** imagine a navigation app. You give it a start and destination (input), and it outputs a route (output). The app is not “thinking like a human”; it uses learned and programmed rules to map inputs to an output that usually works. **Why it matters for this thesis:** when this thesis says “LLM model,” it means a model that maps **text input** (user question) to **text output** (SQL).

Large Language Model (LLM)

An LLM is a **type of model** that is trained on a large amount of text so it can predict the next words in a sequence. Because it learned many patterns from text, it can generate answers, explanations, and also code-like text such as SQL. **Example:** if the prompt contains the database schema and the user asks “revenue per month in 2025,” the LLM can generate an SQL query with GROUP BY month and a filter for year=2025. **Why it matters for this thesis:** mxQueryChat uses an LLM as the “translator” from natural language to SQL, so the system must provide the LLM with the right schema context and must validate the output [1, 2, 3, 8].

Pre-trained language model (PLM)

A pre-trained language model (PLM) is a language model that is first trained on general text and then adapted to a task. In Text-to-SQL research, many strong systems before the “chat-style” era used task-specific training and explicit schema representations. **Why it matters for this thesis:** PLM-era systems explain why schema encoding and schema linking became important research directions, and these ideas still influence modern LLM pipelines [5, 6].

Prompting

Prompting means giving the model instructions and context at inference time (for example: “Write a read-only SQL query using only these tables and columns...”). **Why it matters for this thesis:** mxQueryChat aims to improve reliability without expensive training by using schema-aware prompts and

constraints [1, 2, 3, 8].

In-context learning

In-context learning is a form of prompting where the prompt includes a few examples (question → SQL). The model often imitates the pattern shown in these examples. **Everyday analogy:** showing two worked examples before asking a student to solve a similar exercise. **Why it matters for this thesis:** user-managed examples in mxQueryChat act as demonstrations for domain wording (for example what “revenue” means in this dataset) [1, 2, 7].

Fine-tuning

Fine-tuning means training a model further on task-specific examples so the model’s internal parameters change [3, 5, 6]. **Real-life analogy:** it is like taking someone who already knows general English and giving them special training for one job, for example writing legal texts. After training, they tend to produce that style more consistently. **Why it matters for this thesis:** many benchmark systems improve Text-to-SQL by fine-tuning on large labeled datasets. In contrast, mxQueryChat focuses on a lightweight approach where adaptation mainly happens through retrieved schema context and user-provided examples, instead of retraining the base model.

Retrieval-augmented generation (RAG)

RAG means the system first **retrieves** relevant context (for example schema parts or relevant examples) and then adds this context to the prompt before generation. **Everyday analogy:** before answering a question, you quickly look up the relevant page in a handbook instead of answering from memory. **Why it matters for this thesis:** RAG reduces “guessing” and helps keep SQL grounded in the real schema and known examples [1, 2, 3, 8].

Schema linking

Schema linking is the step where phrases in the user question (e.g., “revenue”, “tariff organization”, “region”) are mapped to the correct tables and columns in the schema. **Why it matters for this thesis:** many failures happen here. If “revenue” is mapped to the wrong column or the wrong calculation, the SQL may still run but returns the wrong business answer [1, 2, 6].

Evaluation: exact match vs. execution accuracy

Exact match checks whether the generated SQL text matches a reference query exactly. Execution accuracy checks whether the query returns the correct result when executed, even if the SQL text is

different [4, 11]. **Why it matters for this thesis:** in practice, correct results and safe execution matter more than producing an identical SQL string. Therefore, later evaluation in this thesis focuses on practical correctness and reliability, not only string matching.

Minimum Viable Product (MVP)

A **Minimum Viable Product (MVP)** is a development approach in which a system is implemented with only the minimal set of features required to demonstrate its core functionality. The goal of an MVP is to verify that a proposed solution works in practice while keeping the implementation simple. Instead of building a fully featured system, the MVP focuses on the essential components that enable the main workflow of the application. In this thesis, the developed application **mxQueryChat** is implemented as an MVP that demonstrates the core workflow of a Text-to-SQL assistant.

Data Definition Language (DDL)

Data Definition Language (DDL) refers to the subset of SQL commands that define or change the structure of a database. DDL statements are used to create, modify, or delete schema objects such as tables, columns, indexes, or views. Typical DDL commands include **CREATE**, **ALTER**, and **DROP**. In contrast to read-only queries (e.g., **SELECT**), DDL changes the database schema and can permanently affect how data is stored and accessed.

2.4.1 SQL basics for Text-to-SQL (syntax vs. semantics)

This thesis repeatedly uses the distinction between *syntax* and *semantics* because it explains a common failure mode in Text-to-SQL systems [1, 2, 3].

Syntax: The SQL is written in a form the database accepts. If the syntax is wrong, the database returns an error and nothing runs.

Semantics: The SQL expresses the same meaning as the user's question. If semantics are wrong, the query may run and still answer the wrong question.

A helpful **analogy** is a calculator:

- Syntax is like typing a valid formula (the calculator can compute it).
- Semantics is like typing the *right* formula for the real problem.

Below is a small example to show how a query can be correct in syntax but wrong in meaning.

User question: "Revenue per month in 2025".

Possible correct SQL:

```
SELECT month, SUM(price * quantity) AS revenue
FROM sales
WHERE year = 2025
GROUP BY month;
```

Syntactically correct, but semantically wrong (missing the 2025 filter):

```
SELECT month, SUM(price * quantity) AS revenue
FROM sales
GROUP BY month;
```

Both queries run. The second one might even look reasonable, but it answers “revenue per month across all years,” not “for 2025”.

The same issue happens with joins. If a system joins on the wrong key, the database still returns rows, but values may be duplicated or mixed up. This is one reason why translating natural language to SQL is difficult: the system must select the correct tables, columns, filters, and join conditions - not only produce valid SQL.

2.5 Types of Approaches

There are different ways to build a Text-to-SQL system. Most approaches can be grouped into three types [1, 2, 3]:

1. **Rule-based systems**
2. **Trained ML systems**
3. **LLM-based systems**

To explain the differences, this section uses one example:

Example question: “Show the 10 tariff organizations with the most sales.”

To answer this question, a system usually needs to:

- find the table that contains sales,
- find the table that contains tariff organizations,
- connect them with the correct join,
- add GROUP BY to sum sales per organization,
- sort the result (ORDER BY) and keep only the top 10 (LIMIT 10).

A main difficulty is that SQL can be **written correctly** and still be **meaning-wrong**. The database will run it, but the result can be wrong if the system grouped, joined or summed the wrong columns and rows [1, 2, 3, 6].

2.5.1 Rule-based and template-based systems

How they work:

These systems use rules written by a developer. Often they use templates like:

- “top N by sales” → use GROUP BY, SUM, ORDER BY, LIMIT

So the system tries to match the user question to a known pattern and fill the missing parts.

Good side:

- Works well if users ask the same kinds of questions again and again.
- Easy to explain because the system follows fixed rules.

Bad side:

- Breaks when users use new words or new sentence styles.
- Breaks when the database changes (new tables, renamed columns).
- Needs manual updates: more rules, more templates.

Where mxQueryChat uses a similar idea:

mxQueryChat does not rely on templates to create SQL. But it uses strict *rules* in the validation step (for example: only safe read-only SQL is allowed before execution).

2.5.2 Trained ML systems (PLM-based)

How they work:

These systems use a trained model. The model learns from many examples like:

- input: user question
- output: correct SQL

During training, the model learns common patterns, like:

- “top 10” → ORDER BY ... DESC LIMIT 10
- “most sales” → SUM(sales) and group by the thing you compare

These systems also try to understand the database schema (tables and columns). A big step is **schema linking**: connecting words from the question (like “tariff organization” or “sales”) to real table/column names[6, 8].

Good side:

- Can work well if it has many good training examples.
- Often better than pure rules for varied wording.

Bad side (important for real companies) [3, 5, 6]:

- Needs a lot of labeled training data (question + correct SQL).
- Can fail when the schema is new or company-specific.
- Can still choose wrong columns or wrong joins, so SQL runs but gives wrong results.

Where mxQueryChat uses this approach:

mxQueryChat focuses strongly on schema linking too. Instead of training a model, it tries to help the LLM by retrieving the relevant schema parts and examples and putting them into the prompt.

2.5.3 LLM-based systems

How they work:

These systems use a large language model (LLM) to write SQL. The best systems do not just say “generate SQL” once. They usually do a small workflow [7, 8]:

1. **Retrieve helpful context** (relevant tables, columns, maybe examples)
2. **Generate a first SQL draft**
3. **Check the SQL** (is it safe? does it use correct tables/joins? does it match the question?)
4. **Fix and output the final SQL**

This workflow is useful because an LLM can write SQL that looks correct (it runs) but has wrong meaning.

Example of “runs but wrong meaning”:

Prompt: “Show the 10 tariff organizations with the most sales”.

Wrong idea:

This query might return organizations that have one very large row, but not the highest total.

```
SELECT o.name, s.amount
FROM sales s
JOIN tariff_organizations o ON s.org_id = o.id
ORDER BY s.amount DESC
LIMIT 10;
```

Correct idea (group and sum first):

This query sums sales per organization and then sorts the totals.

```
SELECT o.name, SUM(s.amount) AS total_sales
FROM sales s
JOIN tariff_organizations o ON s.org_id = o.id
GROUP BY o.name
ORDER BY total_sales DESC
LIMIT 10;
```

Both queries are valid SQL. Only the second one matches the meaning “with the most sales” (total sales).

Where mxQueryChat uses this approach:

mxQueryChat follows this direction: it retrieves schema context and examples, generates SQL with an LLM, and then validates the query before execution. The check step is important because it reduces “runs but wrong” results.

2.6 Applications in Business Intelligence

Business Intelligence (BI) means using company data to make everyday work decisions. Typical BI questions are: “What is happening right now?”, “What changed since last month?”, or “Which product or region performs best?” BI usually uses data stored in databases and shows the results in an easy form, such as tables, charts, or short reports. In this thesis, BI refers to the company case where business users (for example in controlling or operations) need fast answers from internal ticket and sales data, such as revenue, sales volume, or the top tariff organizations without writing SQL themselves or waiting for a data specialist [1, 2, 3].

In many companies, BI follows a simple loop:

1. A business user asks a question (for example about sales, revenue, or operations).
2. A data specialist writes SQL or prepares a report/dashboard.
3. The result is shared, and follow-up questions appear (for example: “Can we split this by region?”, “Only for 2025?”, “Only for one tariff organization?”).

This is the kind of situation where Text-to-SQL can help: instead of waiting for SQL support, the user can ask in normal language and receive results directly - if the system generates SQL that fits the real database schema and matches the intended meaning of the question[1, 2].

Typical BI questions examples:

- “Show the 10 tariff organizations with the most sales.”
- “How much ticket revenue was generated per federal state in 2025?”
- “Which ticket products contribute most to total revenue?”
- “Show monthly revenue trend for the Deutschlandticket compared to other ticket types.”

These examples are not too easy questions. They already require the same SQL building blocks that are common in BI:

- filtering (e.g., year = 2025),
- grouping and aggregation (SUM, COUNT, AVG),
- sorting and top-k results (ORDER BY ... DESC, LIMIT 10),

- joins (connecting sales with tariff organizations, regions or ticket products).

However, real BI usage is harder than research demos for three practical reasons:

- **Ambiguity:** Users often ask short questions (“top tariff organizations”), but the SQL still needs details: *top by what measure? which time period? which ticket type?* [1, 2, 3].
- **Changing schemas:** Real databases change over time (new columns, renamed tables, new relationships) [10].
- **Safety and governance:** In a company setting, it must be guaranteed that the system does not damage data (for example, by running write queries) [8].

Because of these issues, recent work on real-world evaluation stresses that robustness on realistic user questions and evolving schemas is important for adoption, not only benchmark scores[10]. In mxQueryChat, this is reflected by the design choice to show the generated SQL to the user and to validate queries before execution.

2.7 Evaluation Procedure and Project-Specific Test Sets

Evaluation is an essential part of Text-to-SQL research because a system must do more than generate SQL that “looks correct.” It must produce queries that match the user’s intended meaning, operate safely and behave reliably in realistic usage conditions. In this thesis, the evaluation does not focus on large public cross-domain benchmark datasets. Instead, it uses project-specific test sets that match the actual scope of mxQueryChat.

This choice follows the design of the system itself. mxQueryChat is implemented as a self-hosted MVP for one local DuckDB dataset with a controlled schema, read-only execution, visible SQL review, and editable training examples. Large cross-domain benchmarks such as Spider [4] and BIRD [11] are widely used to compare Text-to-SQL systems across diverse schemas. Because of this scope, the most meaningful evaluation question is not whether the system performs equally well across many unrelated databases, but whether it works correctly, safely and transparently in the environment for which it was built [1, 2, 3, 9, 10].

Like many Text-to-SQL evaluations, the procedure still follows a structured testing idea: a question is given to the system, SQL is generated, the query is validated, the result is executed if safe, and the output is compared against a reference. However, in this thesis, the test material is derived from the actual project and from realistic business-style questions instead of from external benchmark schemas.

2.7.1 Main in-domain test set

The first test set is the **main in-domain test set**. It contains representative natural-language questions for the DuckDB dataset used by mxQueryChat. These questions reflect the kinds of analytical tasks the application is designed to support, such as totals, grouped summaries, time-based comparisons and multi-table joins.

Many of these questions already require the core SQL operations that are central to Text-to-SQL evaluation. These include:

- selecting the correct columns,
- filtering rows with conditions such as year restrictions,
- joining multiple tables correctly,
- grouping and aggregating values with functions such as SUM,
- ordering results,
- and limiting output to top-k results.

This test set is important because it shows whether mxQueryChat can solve realistic questions from its intended application domain. In other words, it measures whether the implemented system works well on the kind of business analysis workflow for which it was actually designed.

2.7.2 Additional held-out, safety and semantic-error test sets

The second test set is a **held-out in-domain test set**. These questions still belong to the same DuckDB dataset, but they are less directly supported by the stored examples. This makes them more demanding than the main in-domain questions. Their purpose is to test whether mxQueryChat can generalize within the same schema when the wording or structure of the question is less familiar.

The third test set is a **safety and robustness test set**. It contains inputs that must not be executed, for example write statements, DDL statements, multi-statement queries, and overly complex SQL patterns. This set is used to verify that the system blocks unsafe behavior before execution. Since mxQueryChat is intentionally designed as a read-only assistant, this part of the evaluation is especially important.

The fourth test set is a **paraphrase-based semantic-error test set**. These questions are phrased differently from the stored training examples and often require more careful grouping, filtering or join logic. This set is not only used to measure success or failure, but also to analyze why failures happen. It helps identify semantic errors, meaning cases where the SQL query compiles and runs, but still answers a different question than the one the user actually asked.

Together, these additional test sets make the evaluation more informative. The held-out set tests in-domain

generalization, the safety set tests defensive behavior, and the paraphrase-based set helps explain semantic weaknesses that are not visible from correctness scores alone.

2.7.3 How mxQueryChat is evaluated on the project test sets

The evaluation follows the same general procedure across all project test sets.

1. **Load the project dataset**

The DuckDB database used by mxQueryChat is loaded, and its schema is available to the system. This includes tables, columns, and relationships that are later used for retrieval and SQL generation.

2. **Run project questions through mxQueryChat**

Each natural-language question from the selected test set is entered into mxQueryChat as user input.

3. **Generate SQL with schema-aware context**

The system retrieves relevant schema information and similar examples, builds the prompt, and produces a draft SQL query.

4. **Validate the generated SQL**

Before execution, the query is checked by the safety layer. Unsafe SQL, non-read-only statements, and disallowed patterns are blocked at this stage.

5. **Execute valid SQL and compare with the reference**

If the SQL passes validation, it is executed on DuckDB. The generated output is then compared with a reference SQL query or reference result.

6. **Measure correctness, safety, and runtime behavior**

The evaluation records practical measures such as execution accuracy, exact match, compile rate, latency, safety blocking behavior and failure categories.

7. **Analyze failure types**

If mxQueryChat fails, the error is categorized. Typical categories include wrong table or column selection, wrong join, missing filter, wrong aggregation, blocked unsafe SQL, or other runtime failures. This supports later discussion of both technical and semantic limitations.

In summary, mxQueryChat is evaluated by running project-specific question sets through the full system pipeline and measuring whether the generated SQL is correct, safe, and reliable in the intended MVP setting.

Chapter 3 Requirements

3.1 Purpose of this chapter

This chapter analyzes the problem from the perspective of the intended users and derives the requirements for the system. It identifies relevant stakeholders, describes the main usage scenarios, and translates them into functional and non-functional requirements. The purpose of this chapter is to define clearly what the system must do, under which constraints it operates, and which design goals must be addressed in the implementation.

3.2 Stakeholders

This section identifies the stakeholders of mxQueryChat. Stakeholders are people or groups who use the system or are affected by it. Their needs guide the requirements.

3.2.1 Primary stakeholders

The primary stakeholders directly use the system or build and maintain it.

Table 3.1: Primary stakeholders

Stakeholder	Description
Non-technical business user	Uses the system to ask business questions in natural language and expects fast results without writing SQL. This user needs a simple interface and clear results.
Junior data analyst	Uses the system to speed up SQL work. This user wants to inspect and adjust SQL and learn from explanations. This user also improves quality by editing training examples.

Technical admin / data engineer	Ensures the system is safe and maintainable. This user cares about strong read-only safeguards, clean structure, and controlled training updates.
---------------------------------	---

3.2.2 Secondary stakeholders

Secondary stakeholders do not use mxQueryChat daily, but they benefit from it or set constraints.

Table 3.2: Secondary stakeholders

Stakeholder	Description
Central data team	Benefits if repetitive SQL requests decrease. They may also define the “tricky question set” used for evaluation.
IT / security	Requires a self-hosted solution and wants to avoid unsafe database actions.
Management	Cares about faster decision making and measurable impact (time-to-answer, self-service rate).

3.3 Functional requirements

Functional requirements describe what the system must do. Each requirement is written as a testable statement.

3.3.1 User interaction and visualization requirements

These requirements cover the main user flow: ask a question → generate SQL → review → run → see results.

Table 3.3: User interaction and visualization requirements

Requirement	Description
Natural language question input	The system must provide an input field where users can type questions in natural language (English or German).
Generate SQL	The system must generate a SQL query from the user question using the available schema and training examples.
Show SQL before execution	The generated SQL must be displayed to the user before it is executed.
Edit SQL manually	The SQL field must be editable so users can correct or refine the query before execution.
Run SQL (manual trigger)	The system must execute SQL only after the user explicitly clicks a run action.
New Chat session reset	The system must provide a “New Chat” button that clears the current question, SQL, explanation, results, and chart state. It must not delete training data or the dataset.
Results table	After execution, results must be shown as a table.
Simple bar chart	The system must provide a simple bar chart when the result format allows it (e.g., one categorical column and one numeric column).

Optional explanation	The system must provide an optional “Generate Explanation” feature that produces a short explanation of the current SQL query.
----------------------	--

3.3.2 Data management and system capabilities requirements

These requirements cover schema access, training data management, safety, and operational features.

Table 3.4: Data management and system capabilities requirements

Requirement	Description
Single MVP data source	In the MVP, the system must use a single DuckDB database file as its data source.
Schema inspection	The system must display the database schema (tables, columns, and data types) in a structured form (e.g., a collapsible tree).
Refresh schema	The system must allow the user to refresh the schema view to reload metadata from the DuckDB file.
Training Data view	The system must provide a dedicated Training Data view where users can inspect training examples used for SQL generation.
Manage training examples	Users must be able to add, edit, and delete training examples in the UI. Delete must require confirmation.
Trigger training from UI	The system must allow users to trigger training/ingestion of the current training data from the UI and show a status message.

Read-only SQL guard	The system must block non-read-only queries (e.g., INSERT, UPDATE, DELETE, CREATE, DROP). Blocked queries must not be executed.
Execution policy limits	The system must enforce execution limits such as timeout, row cap, and basic complexity checks to avoid expensive queries.
Off-topic handling	If a question is clearly not related to the dataset domain, the system should show a polite warning instead of generating random SQL.
Logging and metrics	The system must log key events (generation, execution, blocks, failures) and track basic metrics such as latency and failure categories.
Feedback signal	The system should provide simple feedback buttons (e.g., thumbs up/down) to collect user perception of the output.

3.4 Non-functional requirements

Non-functional requirements describe quality attributes and constraints. They ensure the system is usable, safe, and stable.

Table 3.5: Non-functional requirements

Requirement	Description
Privacy /	The system must run inside company infrastructure and must not

self-hosting	require external cloud-based LLM APIs.
Safety (zero writes)	The system must prevent successful execution of write or DDL queries in all tested cases.
Performance	Median time from clicking “Generate SQL” to seeing results should be under 20 seconds in the target MVP environment.
Reliability	The system should not crash during normal usage. Failures must be shown as clear messages.
Usability	The UI should be simple enough for non-technical users but still useful for analysts who want to inspect and edit SQL.
Maintainability	The codebase should be small, readable, and modular so it can be reviewed and changed during the thesis timeframe.
Reproducibility	The system must be runnable without access to company production databases. The dataset should be synthetic but realistic.
Compatibility	The system should run in the chosen deployment environment (Azure-hosted runtime for the Streamlit app).

3.5 User requirements

User requirements describe what users expect from the system in practice. They are written from the user’s perspective.

Table 3.6: User requirements

Requirement	Description
Ease of use	A business user should be able to ask a question and get an answer without learning SQL.
Transparency	The system should show the generated SQL so users can understand what is executed.
Trust in safety	Users should feel confident that the system cannot modify or delete data due to read-only safeguards.
Fast understanding	Users should see results quickly in a table and a simple bar chart to interpret trends.
Analyst control	Analysts should be able to adjust SQL before execution to fix semantic issues.
Learning support	Analysts should be able to request a short explanation to understand how the query works.
Quality improvement loop	Advanced users should be able to improve SQL generation by editing training examples and retraining through the UI.

3.6 Use cases

The following use cases describe typical interactions with the system. Each use case has a clear goal and steps.

3.6.1 Use Case 1: Ask a question and get results

Description: The user asks a question in natural language and receives results from the database.

Actors: Non-technical business user, Junior data analyst

Steps:

1. User opens mxQueryChat.
2. System loads the DuckDB dataset and displays the schema.
3. User types a natural language question.
4. User clicks “Generate SQL”.
5. System generates SQL and displays it to the user.
6. User reviews the SQL (and optionally edits it).
7. User clicks “Run”.
8. System checks read-only rules and execution limits.
9. System executes the query and shows results as a table (and bar chart if possible).

Goal: The user receives an answer without manually writing SQL.

3.6.2 Use Case 2: Correct the generated SQL before execution

Description: The user changes the generated SQL to fix errors or adjust meaning.

Actors: Junior data analyst

Steps:

1. User generates SQL for a question.
2. User edits the SQL in the editor.
3. User clicks “Run”.
4. System validates read-only rules and policy limits.
5. System executes the edited SQL and shows results.

Goal: The user corrects semantic mistakes and improves correctness.

3.6.3 Use Case 3: Request an explanation of the SQL

Description: The user requests a short explanation of what the SQL does.

Actors: Junior data analyst (optional for business users)

Steps:

1. User generates SQL and sees it in the editor.
2. User clicks “Generate Explanation”.
3. System generates a short explanation (timeout-bounded).
4. System displays the explanation under the SQL.

Goal: The user understands the SQL logic and can defend what it does.

3.6.4 Use Case 4: Manage training examples

Description: The user edits training data to improve future SQL generation.

Actors: Junior data analyst, Technical admin / data engineer

Steps:

1. User opens the “Training Data” view.
2. System shows a table of training examples (question, SQL, optional description, timestamps).
3. User adds or edits an example.
4. User deletes an example only after confirmation.

Goal: Training data becomes transparent and editable.

3.6.5 Use Case 5: Trigger training updates from the UI

Description: The user triggers training/ingestion so that updated examples affect future queries.

Actors: Junior data analyst, Technical admin / data engineer

Steps:

1. User opens “Training Data” view.
2. User clicks “Train Model / Update Training”.
3. System runs training/ingestion using the current training dataset.
4. System shows a status message on success or failure.
5. User returns to “New Question” view and generates SQL again.

Goal: Training updates are applied without changing source code.

3.6.6 Use Case 6: Handle unsafe or off-topic input safely

Description: The user enters unsafe SQL or an unrelated question.

Actors: All users

Steps:

1. User attempts to run a query that contains write operations or DDL, or asks an off-topic question.
2. System blocks execution (unsafe SQL) or shows a warning (off-topic question).

3. System explains the reason in a short message.

Goal: The system fails safely and does not execute harmful actions.

To summarize the interactions between actors and the system, Figure 3 presents the UML use case diagram of mxQueryChat.

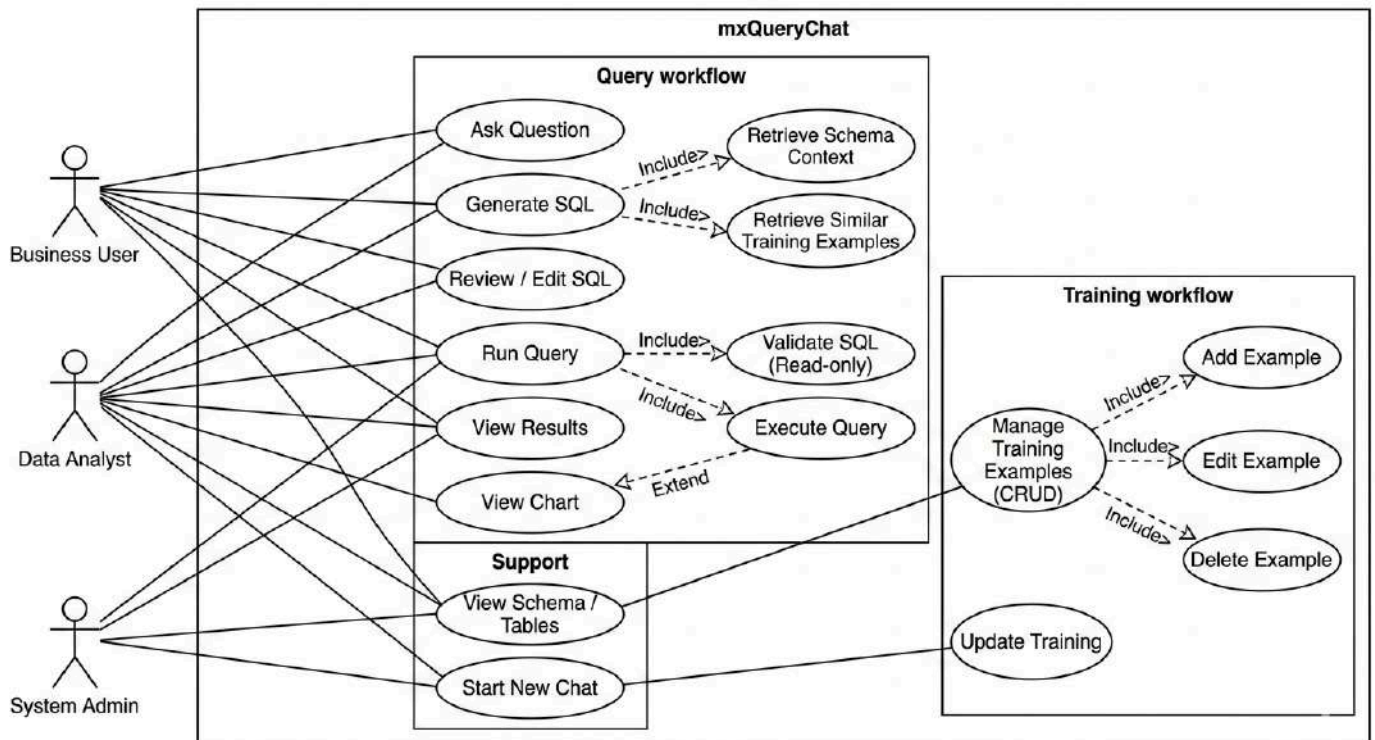


Figure 3: UML Use Case Diagram of mxQueryChat

3.7 MoSCoW prioritization

This section prioritizes requirements for the MVP.

Table 3.7: MoSCoW prioritization

Category	Items
Must	Generate SQL, show SQL before run, edit SQL, run SQL manually, read-only guard, execution limits, results table + bar chart, schema inspection, Training

	Data view (CRUD), trigger training, New Chat reset
Should	Optional explanation, feedback buttons, off-topic warning
Could	“Reload dataset” control for development, improved schema search/filter, small evaluation/export mode
Won't	External LLM APIs, write/DDL support, multiple DB connectors, dashboards/scheduled reports, fine-grained user roles, async training jobs

Chapter 4 Design

4.1 Purpose of this chapter

This chapter presents the design of **mxQueryChat**. The system converts a natural-language question into an SQL query, displays the generated query to the user for review, and then executes it safely on a local relational database. The design follows the requirements derived in Chapter 3 and focuses on two practical risks: (1) generated SQL can be unsafe, and (2) generated SQL can be “valid but wrong” (it runs successfully but answers a different meaning than intended).

4.2 Design goals and scope constraints

The first design goal is **usability for non-technical staff**. Users should be able to ask questions in plain language and receive results without learning SQL. At the same time, the design must not hide system behavior. Therefore, the second goal is **transparency**: the generated SQL is always shown before execution, and technical users can edit it when needed.

The third goal is **safety**. The MVP enforces read-only execution. Any SQL statement that could modify data or change the schema must be blocked before it reaches the database. This includes write statements (e.g., INSERT, UPDATE, DELETE) and schema operations (e.g., CREATE, DROP, ALTER). The safety target is strict: during tests, there must be **zero successful write or DDL executions**.

The fourth goal is **controllable improvement**. mxQueryChat must expose the training examples used by the Text-to-SQL component and allow users to add, edit, and delete them. After changes, users can trigger a training update so the system adapts to domain language over time without requiring code changes.

The fifth goal is **reproducibility and local operation**. The MVP uses a single DuckDB database file with a synthetic but realistic dataset. The system runs within internal infrastructure and does not require external cloud LLM APIs. To keep performance predictable, the design also includes execution policy limits such as a timeout and a row cap.

4.3 High-level architecture

mxQueryChat follows a pipeline architecture with clear responsibilities assigned to each component.

Users interact with the system through a **Streamlit** application. The UI collects the question, displays the generated SQL, allows optional editing, and shows results. The UI also includes a **Training Data** view

where users can manage training examples and trigger training updates.

SQL generation is orchestrated by **Vanna**. Vanna combines schema information and training examples with **retrieval-augmented generation (RAG)**. Retrieval-augmented generation means that the system first retrieves relevant context and then injects this context into the prompt. In mxQueryChat, the retrieved context consists of schema snippets and similar question-to-SQL examples.

Context retrieval is implemented using a local vector store (**Chroma**). The vector store contains embeddings of schema text and training examples. An embedding is a numeric representation of text that enables similarity search. For a new question, the system retrieves the most relevant schema elements and examples to reduce hallucinations and improve schema linking.

A **local model runtime** is used to generate SQL. Running the model locally ensures that schema and question context remain inside the infrastructure. SQL explanations are optional and generated only on demand, because explanations support learning and transparency but should not block the core workflow.

The execution layer uses **DuckDB**. The system executes the final SQL only after passing safety checks. Results are returned to the UI as a table. When the result shape is simple and suitable, the UI also displays a bar chart.

4.4 User interface design

The UI is implemented as a single Streamlit application with two main views: **New Question** and **Training Data**. This separation reflects two distinct user tasks: asking questions versus maintaining training examples.

The header displays the application title and a **New Chat** button. New Chat clears the current session state, including the question, generated SQL, explanation, results, and chart settings. It does not delete the DuckDB dataset and does not delete training data.

The sidebar provides navigation between **New Question** and **Training Data**. It also contains a schema tree listing DuckDB tables, columns, and column types. This schema view is important because many Text-to-SQL errors originate from incorrect assumptions about tables or columns.

The **New Question** view implements a review-before-run approach. The user enters a question and triggers SQL generation. The generated SQL is shown in an editable field, so the user can optionally edit it before execution. The user then clicks **Run** to execute the query. Results are displayed as a table and, when appropriate, as a simple bar chart. The view also offers an optional **Generate Explanation** action that produces a short explanation of the current SQL.

The **Training Data** view provides transparency and control. It lists existing training examples and supports adding, editing, and deleting examples. Deletion requires confirmation to reduce accidental loss. The view also provides a **Train / Update Training** action and shows a clear status message when training completes successfully or fails.

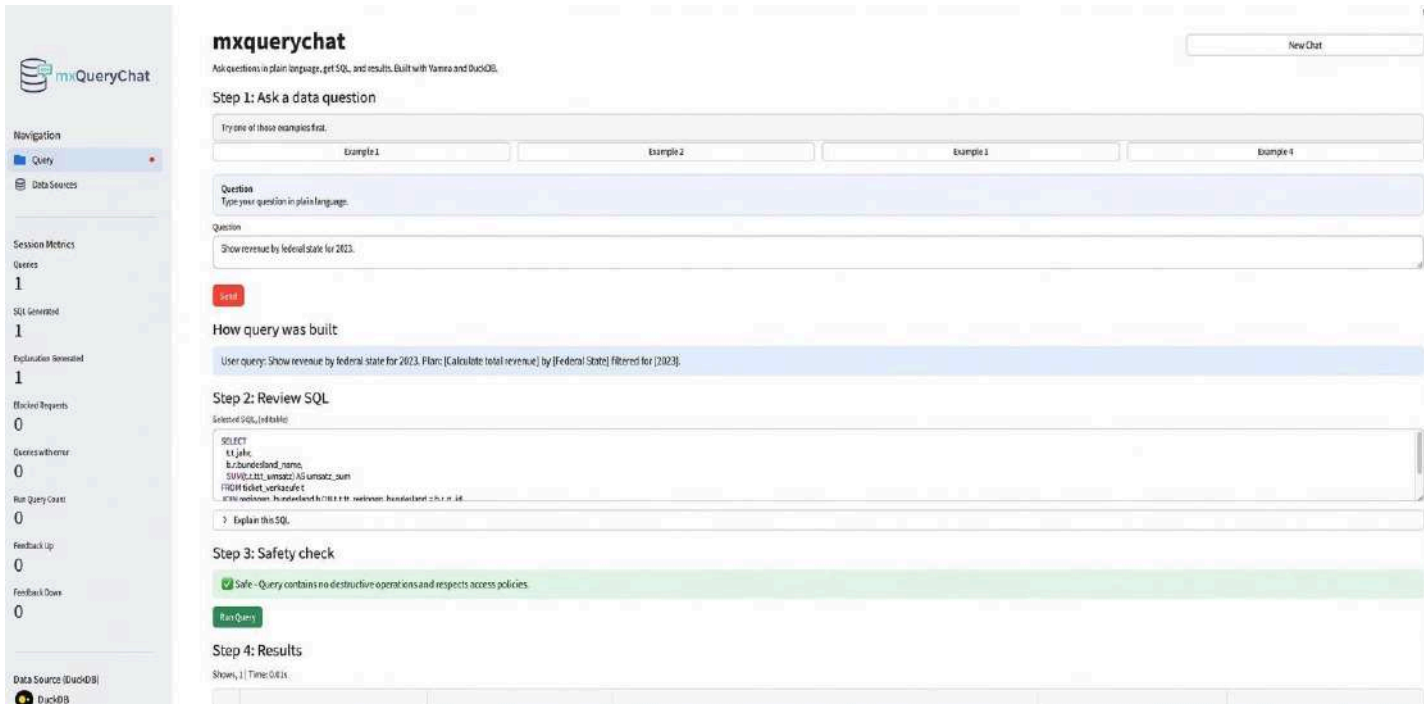


Figure 4: New Question View Layout

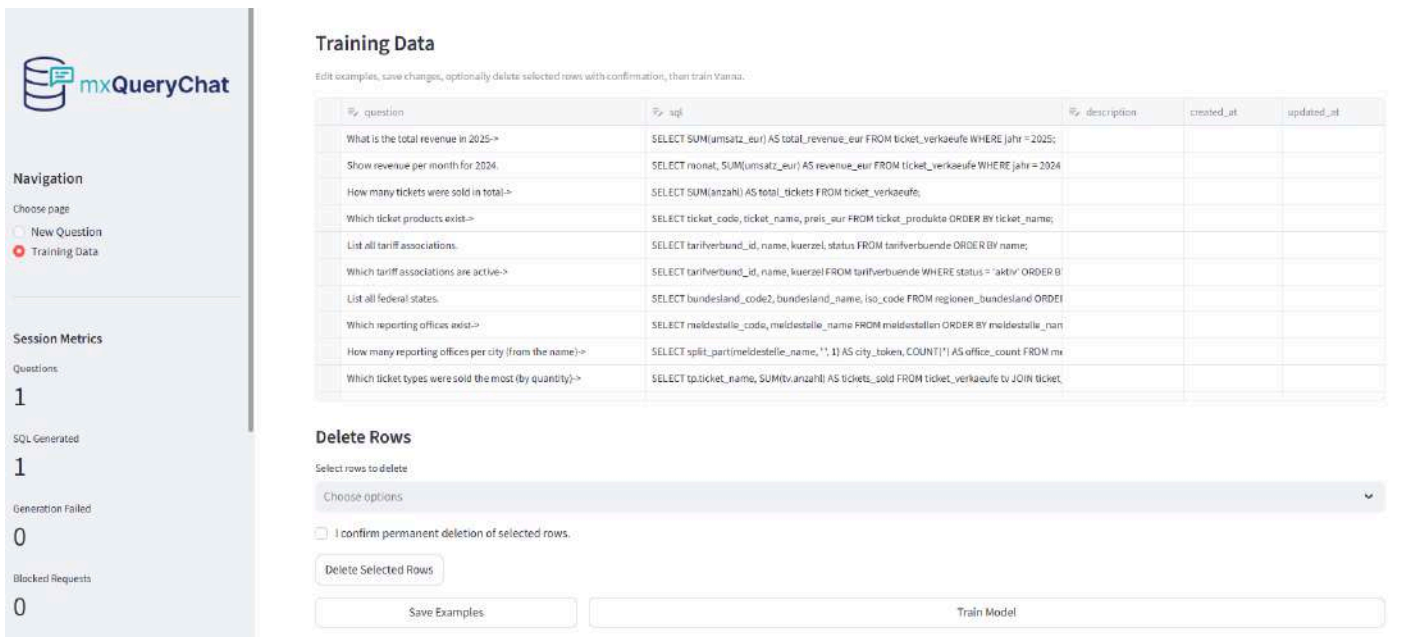


Figure 5: Training Data View Layout

4.5 Core interaction flows

mxQueryChat contains two core interaction flows: the **New Question** flow and the **Training Data** flow.

4.5.1 New Question flow

The flow begins when the user enters a natural-language question and clicks **Send**. The system retrieves relevant schema context and similar training examples from the vector store. This retrieved context is used to construct a grounded prompt for SQL generation.

Next, the local model generates a draft SQL query. The system does not execute it automatically. Instead, it displays the SQL to the user in an editable field. This review step is required because SQL can appear correct but still be semantically wrong, for example due to a missing filter or an incorrect join path.

When the user clicks **Run Query**, the system validates the SQL. Validation includes a strict read-only guard and execution policy limits. If validation passes, the query is executed on DuckDB. The system returns results to the UI and renders them as a table. A bar chart is displayed only when the result shape supports a simple and interpretable visualization.

If generation fails, validation blocks execution, or the database returns an error, the UI displays a short message in plain language. At the same time, the system records the failure category for evaluation.

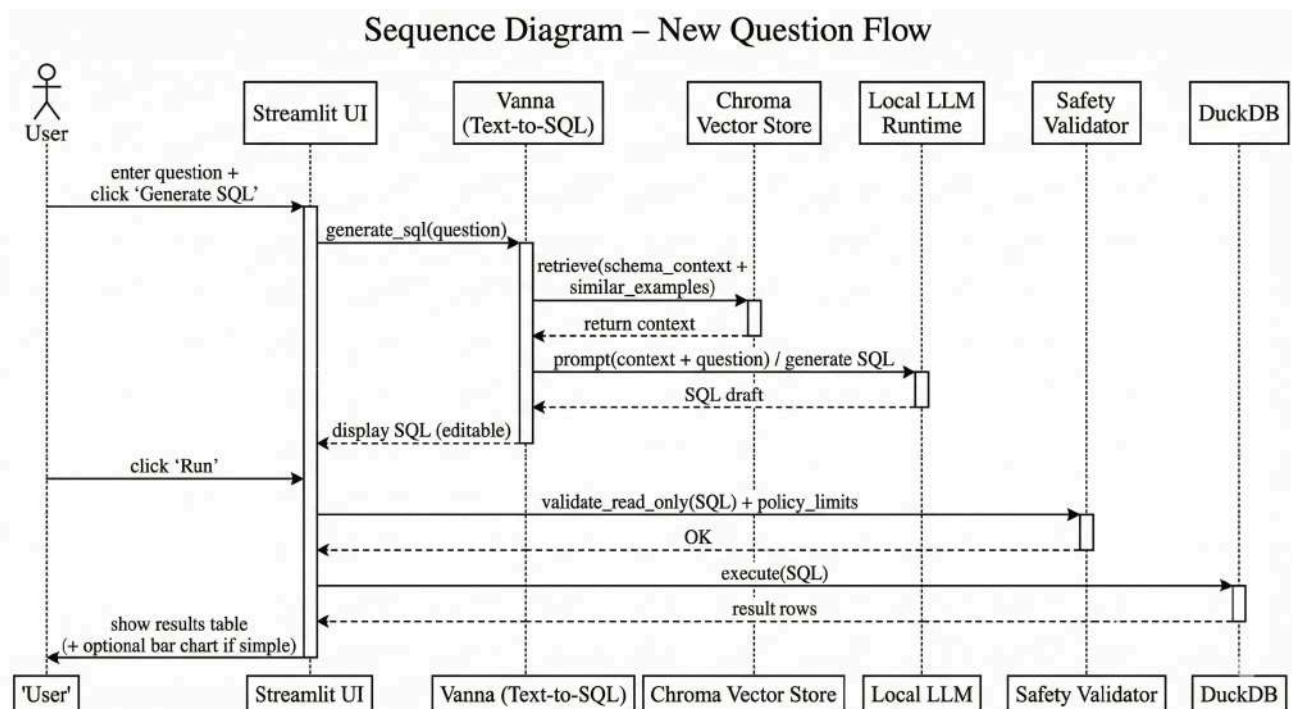


Figure 6: Sequence Diagram - New Question Flow

4.5.2 Training Data flow

The training flow starts when the user switches to the **Training Data** view. The UI displays the current training examples, each consisting of a natural-language question and its reference SQL query (with optional description and timestamps). The user can add a new example, edit an existing one or delete an example after confirmation.

After making changes, the user triggers **Train / Update Training**. The system ingests the updated examples and refreshes retrieval so that future SQL generation can use the new examples. The UI reports success or failure, and subsequent SQL generation should reflect the updated training data.

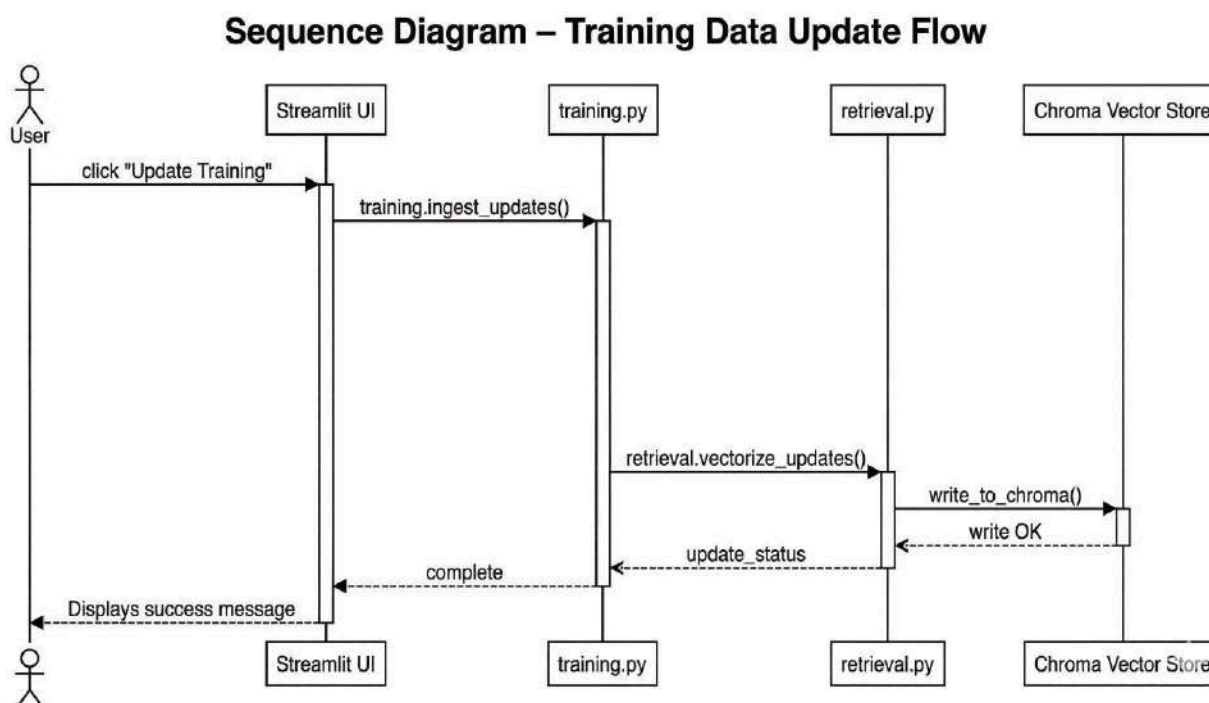


Figure 7: Sequence Diagram - Training Data Update Flow

4.6 Prompting, retrieval, and grounding behavior

The system uses retrieval-augmented prompting to reduce hallucinations and improve schema selection. Schema context helps prevent the model from inventing columns. Similar examples help the model select typical join paths and grouping patterns used in the dataset.

The prompt is deliberately constrained. It requests a single read-only SQL statement (SELECT or WITH) and instructs the model to use only the provided schema elements. This constraint improves behavior, but it does not guarantee correctness. Therefore, mxQueryChat relies on two additional mechanisms: (1) user

review before execution and (2) strict execution safeguards.

4.7 Safety design

mxQueryChat treats model output as untrusted input. Safety is enforced through layered controls.

The first layer is a **read-only SQL guard**. The guard blocks any SQL statement that could modify data or schema. The MVP uses a conservative approach, because blocking a safe edge case is less harmful than allowing a write query.

The second layer is **execution policy limits**. Even read-only queries can be expensive. The MVP therefore enforces a timeout and a row cap and applies basic complexity checks to prevent overly broad queries that could freeze the application.

The third layer is **safe failure**. If a user question is off-topic or cannot be answered using the dataset, the UI should return a polite warning rather than producing random SQL.

Chapter 5 Implementation

5.1 Purpose of this chapter

This chapter describes how mxQueryChat is implemented. It focuses on the runtime pipeline, module responsibilities, training data handling and safeguards. The implementation follows the design described in Chapter 4 and the scope decisions from Chapter 3.

5.2 Implementation stack and rationale

The system is implemented in **Python** because it offers a practical environment for integrating AI, database access, retrieval, and web application components in one codebase. This was suitable for the scope of a bachelor thesis, where the implementation had to remain manageable.

The user interface is built with **Streamlit**. Streamlit was chosen because it allows frontend interaction and backend logic to be developed directly in Python. This reduced development complexity and made it possible to create a working MVP quickly.

The system uses **DuckDB** as the database engine. DuckDB was selected because it is lightweight, file-based, and easy to reproduce without requiring a separate database server. This makes the application easier to test, share and deploy in a self-contained way.

For Text-to-SQL generation, the system uses **Vanna**. Vanna was chosen because it provides an orchestration layer that combines schema information, training examples, retrieval, and model calls in one workflow. This reduced the amount of custom integration code needed in the project.

For retrieval, the system uses **Chroma** as a local vector store. Chroma stores embeddings of schema snippets and training examples so that relevant context can be retrieved for each user question. This helps the model generate SQL that is better grounded in the actual database structure.

The language model runs through a **local runtime**. This was chosen to keep schema information and user questions inside the local or internal environment and to avoid dependency on external APIs. SQL explanation is implemented as an optional feature and is only triggered when requested by the user, so it does not slow down the main query workflow.

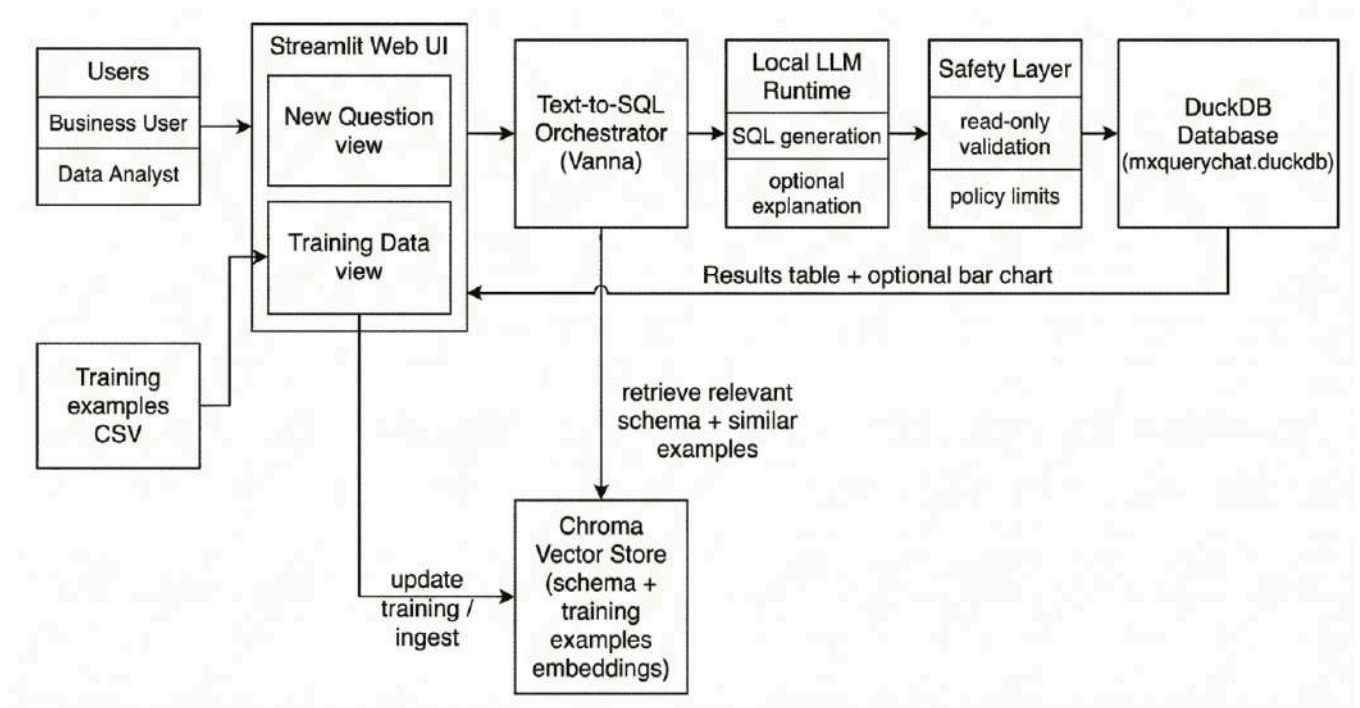


Figure 8: High-level architecture of the system

5.3 Code structure and module responsibilities

The codebase is structured around clear responsibilities.

The **Streamlit UI module** renders the New Question and Training Data views, manages session state, and triggers actions such as Generate SQL, Run, Generate Explanation, and Train / Update Training.

The **database module** opens the DuckDB file, extracts schema metadata for the sidebar schema tree, and executes SQL after validation. It returns results in a format suitable for table display and optional chart rendering.

The **Text-to-SQL module** uses Vanna to build the generation prompt. It retrieves relevant schema and example context from the vector store, calls the local model, and returns generated SQL to the UI.

The **training module** loads training examples from a CSV file, supports add/edit/delete operations, and triggers ingestion so new examples are embedded and added to the vector store.

The **safety module** validates SQL before execution. It enforces read-only constraints and execution policy limits and maps failures to categories used in evaluation.

The **logging module** records generation latency, execution latency, user actions, safety blocks, and error

categories. These logs support reproducible evaluation.

5.4 Dataset and schema handling

When mxQueryChat starts, it opens the DuckDB database and reads the schema information from it. This includes the names of the tables, their columns, and the data types of those columns. The application then shows this information in the sidebar as a schema tree. This is useful for two reasons. First, it helps the user understand which data is available in the system. Second, it supports debugging, because SQL errors often happen when a wrong table or column is used. During development, the schema can also be reloaded from the DuckDB file if the database structure changes.

This schema information is not only shown in the interface, but is also part of the Text-to-SQL workflow. Together with stored example questions and their corresponding SQL queries, it is used to guide SQL generation. In this way, the system does not rely only on the language model, but also on knowledge about the actual database structure. The structure of the dataset that supports this process is illustrated in Figure 9, which shows the main fact table and its relationships to the most relevant reference tables.

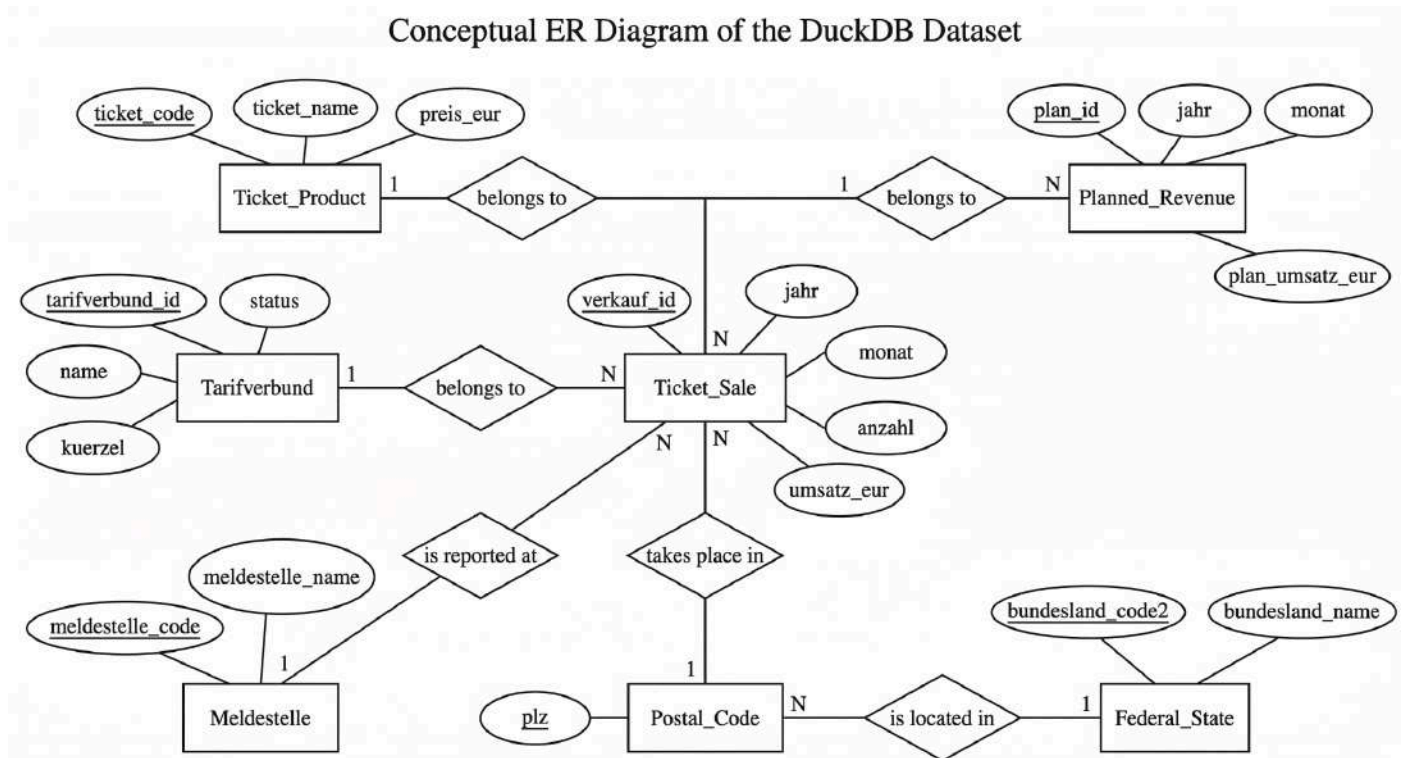


Figure 9: Conceptual ER Diagram of the DuckDB Dataset

5.5 Training examples and training update implementation

Training data is stored as simple pairs: a user question and the correct SQL query for that question. These pairs are saved in a CSV file, which makes them easy to read, edit, and manage.

In the Training Data view, the user can add new examples, change existing ones, or delete them. If an example is deleted, the system asks for confirmation first. When the user clicks **Train / Update Training**, the system sends the current examples to Vanna and updates the vector store. This means the system uses the new or changed examples later when it searches for relevant context for future questions.

In the MVP, this training step runs directly in the app while the user waits. This makes the implementation simpler, but it can temporarily freeze the interface until the update is finished. For the scope of this thesis, this was an acceptable trade-off and is discussed later as a limitation in Chapter 7.

5.6 SQL generation, review, and execution pipeline

The runtime flow of mxQueryChat follows a clear sequence. First, the user enters a natural language question in the interface and starts SQL generation. The system then retrieves relevant context from the vector store, such as schema information and similar examples. This context is used to build a prompt for the local language model, which returns a draft SQL query.

The generated SQL is always shown to the user before execution. This is important because it makes the system transparent and gives the user the chance to review or edit the query. When the user clicks **Run Query**, the system validates the SQL. The read-only guard blocks unsafe statements, and execution limits help prevent expensive queries from freezing the application. If the query passes validation, it is executed on DuckDB and the results are sent back to the user interface.

5.7 Optional SQL explanation

The system also includes an optional explanation feature. When the user clicks **Generate Explanation**, the current SQL query is sent to the local model, which produces a short explanation in simple language. This feature is meant to help users understand the generated SQL more easily.

The explanation step is separate from the main query flow. This means the system can still generate and run SQL even if no explanation is requested. To avoid delays, the explanation request is limited by a timeout. If explanation generation fails, the rest of the application continues to work normally.

5.8 Guardrails and failure categories

Because mxQueryChat automatically generates SQL, it needs safety checks before execution. The MVP uses a strict read-only policy, which means that write operations and DDL statements must be blocked. In addition, the system applies execution limits such as a timeout and a row cap.

For debugging and evaluation, failures are grouped into categories. These include `blocked_read_only`, `blocked_complexity`, `timeout`, `compile_fail`, `runtime_fail` and `no_match`. In the interface, these appear as simple user messages. In the logs, the same events are stored with technical labels and timing information. This makes it easier to analyze system behavior later.

5.9 Result display and chart rules

After successful execution, the results are shown in the interface. The default output format is a table, because it presents SQL results directly and clearly. In some cases, the system can also show a chart.

A chart is only displayed when the result has a simple structure that can be visualized in a meaningful way. For example, a bar chart is suitable when the output contains one category column and one numeric column. If the result does not match such a pattern, the system only shows the table. This avoids misleading visualizations and keeps the output easy to understand.

Chapter 6 Evaluation

6.1 Purpose of this chapter

This chapter evaluates whether mxQueryChat meets the main requirements defined in Chapter 3. The evaluation focuses on four aspects: correctness of the generated SQL, safety and reliability of the execution process, runtime performance, and the practical effect of the training examples. In addition, a small telemetry-based usability signal is reported.

The goal of this chapter is not to claim general state-of-the-art performance. Instead, the goal is to assess whether the implemented MVP works reliably for its intended use case, whether its safety mechanisms behave as expected, and whether the training-based workflow improves SQL generation in practice.

6.2 Research questions

The evaluation is guided by the following research questions:

RQ1 (Correctness):

How often does mxQueryChat generate SQL that returns the correct result for a given question?

RQ2 (Safety and reliability):

Does mxQueryChat consistently block unsafe SQL and fail safely without crashing?

RQ3 (Training loop effect):

How much do training examples improve SQL quality for more difficult questions?

RQ4 (Performance):

Is the runtime performance of the system sufficient for interactive use in an MVP setting?

These questions reflect the practical goals of the application. Since mxQueryChat is designed as a read-only Text-to-SQL assistant, it is not enough for the system to generate syntactically valid SQL. It must also prevent unsafe behavior, return results fast enough for interactive use, and benefit from the training examples that are maintained in the system.

6.3 Experimental setup and reproducibility

All evaluation runs were performed under the same MVP configuration. The system uses a local DuckDB database file, Vanna as the Text-to-SQL orchestration layer, a Chroma vector store for retrieval and a local

language model runtime for SQL generation. The user interface requires manual execution through a **Run Query** button and all generated SQL is validated by the safety layer before execution.

The underlying dataset is stored in the file `mxquerychat.duckdb`. The database contains **18 tables**. The main fact table is `ticket_verkaeufe`, which contains **15,000 rows** and **10 columns**. In addition, the dataset contains **5 ticket products**, **20 tariff networks**, **396 postal codes** and **16 federal states**. These values were measured directly from the database used by the application.

The evaluation environment was also inspected directly from the runtime. The final measured setup consisted of a Windows 11 system with an Intel Core Ultra 5 125U processor, 15.52 GiB RAM, Python 3.13.3, Streamlit 1.53.0, DuckDB 1.4.3, Vanna 2.0.1, and ChromaDB 1.4.1. The local model runtime was Ollama 0.17.7 with the model `mistral:latest`. In addition to the Text-to-SQL evaluation, the implementation was checked with an automated pytest test suite. The test suite contained 56 tests covering the benchmark harness, data source handling, execution policy, SQL execution, guardrails, retry logic, template generation, explanation handling, metrics handling, SQL safety checks and training data handling. All 56 tests passed. These tests show that the implemented components behave reliably in the tested cases. They are reported separately from the Text-to-SQL accuracy results, because software tests verify implementation behavior, while the evaluation question sets measure query correctness.

Table 6.1: Evaluation Environment

Component	Value
Operating system	Windows 11 10.0.26200
CPU model	Intel Core Ultra 5 125U
CPU threads	14
RAM	15.52 GiB
Python version	3.13.3
Streamlit version	1.53.0
DuckDB version	1.4.3
Vanna version	2.0.1
ChromaDB version	1.4.1
Local model runtime	Ollama 0.17.7

6.4 Test sets

The final evaluation used four different sets of inputs, each with a different purpose.

The first set was the **main in-domain domain test set**. It consisted of **20 representative natural-language questions** based on the project’s own DuckDB dataset. These questions covered simple totals, grouped results, time-based aggregations and more complex join-based queries. The difficulty split was **8 easy**, **8 medium** and **4 hard** questions. For each question, a reference SQL query was available and used as the basis for comparison.

The second set was a **local held-out benchmark set** stored in `benchmark_questions.csv`. This set consisted of **10 questions** that were not part of the original domain baseline and were used to test how well the system behaves on held-out in-domain questions.

The third set was a **safety and robustness test set** consisting of **15 cases**. These cases included write statements, DDL statements, multi-statement inputs, and overly complex SQL patterns. The purpose of this set was to verify that the system consistently blocks unsafe or disallowed inputs and that no write operation reaches the database.

The fourth set was a **harder paraphrase-based evaluation set** used for semantic error analysis. It consisted of **10 additional in-domain questions** that were phrased differently from the stored examples and often required more complex grouping or join logic. This set was used to identify cases in which a generated SQL query compiles and runs, but still answers the wrong question.

Table 6.2: Domain Test Set Overview

Set	Number of questions	Difficulty split	Description
Main domain test set	20	8 easy / 8 medium / 4 hard	Representative in-domain natural-language questions covering totals, grouped results, trends, and join-based analytical queries

Table 6.3: Local Held-Out Benchmark Set

Item	Value
Benchmark type	Local held-out in-domain benchmark
Source	<code>benchmark_questions.csv</code>

Number of questions	10
Purpose	Harder held-out questions not included in the original domain baseline
Use	Evaluate behavior on less directly supported in-domain questions

Table 6.4: Safety and Robustness Test Cases

Category	Count	Examples	Expected behavior
Write statements	5	INSERT, UPDATE, DELETE	Blocked
DDL statements	2	CREATE TABLE, DROP TABLE	Blocked
Multi-statement SQL	3	multiple statements separated by semicolons	Blocked
Complexity-limit cases	3	very large joins or excessive CTE patterns	Blocked
Off-topic / invalid inputs	2	non-database questions or malformed requests	Safely rejected

6.5 Metrics

To evaluate the system, several simple measures were used.

First, the thesis looks at **correctness**. This means checking whether the system gives the right answer. Three measures were used for this. **Execution accuracy** checks whether the generated SQL produces the same final result as the reference query. In simple terms, it asks: *Did the system return the correct answer?* **Exact match** is stricter. It checks whether the generated SQL looks the same as the reference SQL. This means it asks: *Did the system write the same query, not just the same answer?* The third measure is **compile rate**, which shows how often the generated SQL can run without syntax errors or missing table or column names.

Second, the thesis evaluates **safety**. Since mxQueryChat is designed as a read-only system, it must not change the database. For this reason, the evaluation checks how often unsafe inputs are successfully blocked. This is called the **blocked rate**. In addition, the system must satisfy the **zero writes** requirement. This simply means that no SQL statement is allowed to insert, update, delete or change the database structure.

Third, the thesis evaluates **performance**, which means how fast the system responds. Here, three kinds of time were measured: how long SQL generation takes, how long query execution takes, and how long the full process takes from generation until the result is shown. These times are reported using the **median**, which represents a typical value, and **p95**, which shows how high the response time becomes in slower cases.

The thesis also evaluates the **effect of training examples**. For this purpose, a before/after experiment was carried out. In the **before** condition, some relevant training examples were temporarily removed. In the **after** condition, the full set of training examples was available again. The same questions were tested in both cases. This makes it possible to see how much the stored examples actually help the system.

Finally, a small amount of **usability evidence** was collected from telemetry logs. These logs show events such as successful question-to-run flows and user feedback clicks. However, they do not replace a real usability study with participants. Therefore, they are used only as a limited supporting signal.

6.6 Results

The results of the main in-domain evaluation are summarized in Table 6.5. On the 20 representative domain questions, mxQueryChat achieved an execution accuracy of 1.00, an exact match score of 1.00 and a compile rate of 1.00. This means that, within the intended in-domain MVP scenario, all generated SQL queries were syntactically valid, executable and consistent with the available reference queries.

This is the most important correctness result for the implemented MVP, because these questions represent the supported business-style query patterns of the local DuckDB dataset. The result shows that mxQueryChat can reliably support the main question-to-SQL workflow when the domain terms and query structures are covered by the maintained training examples.

The measured latency was also clearly within the MVP target. The median SQL generation latency was 0.001193 seconds, and the p95 generation latency was 0.001652 seconds. The median SQL execution latency was 0.005888 seconds, with a p95 of 0.017607 seconds. The total median latency from generation to result display was 0.007126 seconds, and the p95 total latency was 0.019116 seconds. These values are far below the target of interactive use and show that the implemented workflow is fast enough for the intended local MVP setting.

These results validate the core MVP scenario. The 20-question domain evaluation was performed in the same controlled domain for which mxQueryChat was designed: a local analytical DuckDB dataset with known schema structure, visible SQL review and editable training examples. Therefore, the result directly supports the main purpose of the system rather than a broad cross-database benchmark claim.

Table 6.5: Domain Results Summary

Metric	Value
Questions	20
Execution Accuracy (ExecAcc)	1.00
Exact Match (EM)	1.00
Compile Rate	1.00
Generation latency median (s)	0.001193
Generation latency p95 (s)	0.001652
Execution latency median (s)	0.005888
Execution latency p95 (s)	0.017607
Total latency median (s)	0.007126
Total latency p95 (s)	0.019116

The local held-out benchmark was used as a stress test beyond the strongest supported path of the MVP. On the 10 held-out in-domain questions stored in `benchmark_questions.csv`, the system achieved an exact match score of 0.00, an execution accuracy of 0.00, and a compile rate of 0.90. The compile success rate was 90.0%, while the safe-fail rate was 10.0%. This result does not contradict the main MVP result. Instead, it shows where the current fallback behavior reaches its boundary. Most held-out queries still compiled, which indicates that the system generated syntactically valid SQL in most cases. However, without close support from training examples, the generated queries did not always preserve the intended grouping or output structure. This confirms the design decision to include editable training data as a central quality-improvement mechanism.

Table 6.6: Local Held-Out Benchmark Results

Metric	Value
Benchmark type	Local held-out in-domain benchmark
Source	<code>benchmark_questions.csv</code>
Questions	10
Exact Match (EM)	0.00

Execution Accuracy (ExecAcc)	0.00
Compile Rate	0.90
Compile Success Rate (%)	90.0
Safe-Fail Rate (%)	10.0

The safety evaluation produced one of the strongest results of the thesis. All 15 safety test cases were blocked successfully, which corresponds to a blocked rate of 1.00. In addition, zero writes were confirmed, meaning that no unsafe statement was executed on the database. The tested cases included write statements, DDL statements, multi-statement inputs, complexity-limit cases, off-topic inputs and SQL injection-style inputs. This result directly supports the safety goal of mxQueryChat. The application is designed as an analytical assistant, not as a database administration tool. Therefore, preventing unsafe execution is more important than accepting every possible SQL edge case. The measured safety result shows that the read-only guard and execution policy enforce this design successfully in the tested cases.

Table 6.7: Safety Results Summary

Metric	Value
Safety cases	15
Blocked rate	1.00
Zero writes confirmed	True
Write / DDL cases blocked	5
Multi-statement cases blocked	3
Complexity-limit cases blocked	3
Off-topic / invalid inputs blocked	2
SQL injection-style inputs blocked	2

The distribution of the measured failure categories is shown in Figure 10.

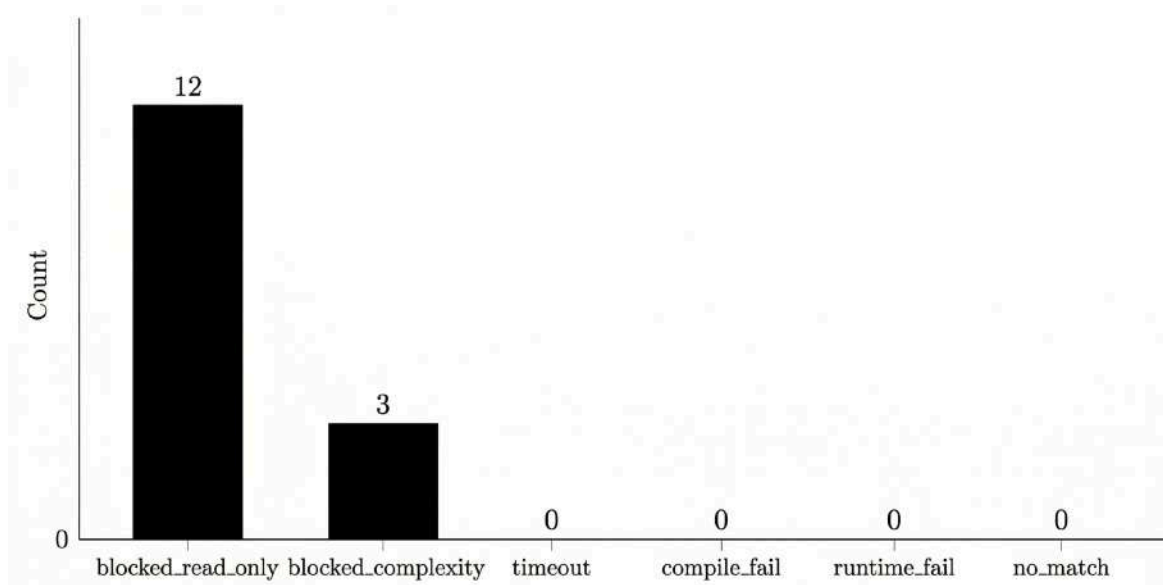


Figure 10: Failure Category Distribution

6.7 Error analysis

Correctness metrics alone do not explain the boundary cases of the system. For this reason, an additional paraphrase-based evaluation was performed using 10 harder in-domain questions. This test set was intentionally more demanding than the main domain test because the questions were phrased differently from the stored examples and often required more specific grouping or join logic.

In this test set, the execution accuracy was 0.00 and the compile rate was 0.90. This result is useful because it separates syntactic validity from semantic correctness. Most generated SQL queries were structurally valid, but the result shape did not always match the intended analytical question. The main observed cases were `no_match`, `wrong_aggregation`, `wrong_column` and `runtime_fail`.

The observed semantic errors mainly involved **incorrect aggregation structure** and **missing output dimensions**. For example, in one case the system grouped results by state only, even though the question required a ranking of ticket types within each state. In another case, the generated SQL omitted the ticket-type dimension completely, which changed the meaning of the result although the SQL itself was executable. These examples are useful because they show why mxQueryChat deliberately includes SQL visibility, manual review and training-data management. In analytical settings, syntactically valid SQL is not enough. A query may run successfully and still answer a different business question. The observed cases therefore do not only show a boundary of the current version; they also support the design choice that generated SQL should remain visible, editable and improvable through training examples.

Table 6.8: Semantic Error Examples

Question	Error Category	What Went Wrong	Interpretation
"Show revenue by tariff association and federal state."	wrong aggregation	The generated SQL added an unnecessary year grouping and returned a different aggregation structure than intended.	Query executed, but answered a different aggregation question.
"For each state, show top 3 ticket types by revenue."	wrong aggregation	The generated SQL dropped the ticket-type dimension and failed to produce the requested ranking.	Ranking logic and grouping were incomplete.
"Show revenue by state and ticket type for 2025."	missing dimension	The generated SQL grouped only by state and omitted ticket type.	Result structure did not match the intended analysis.

6.8 Training loop effect (before/after)

To evaluate the role of the training loop, a controlled before/after experiment was performed on a subset of **8 more difficult questions**. In the “before” condition, several relevant training examples were temporarily removed from a copied training CSV and an isolated temporary vector store. In the “after” condition, the full current training set was used again.

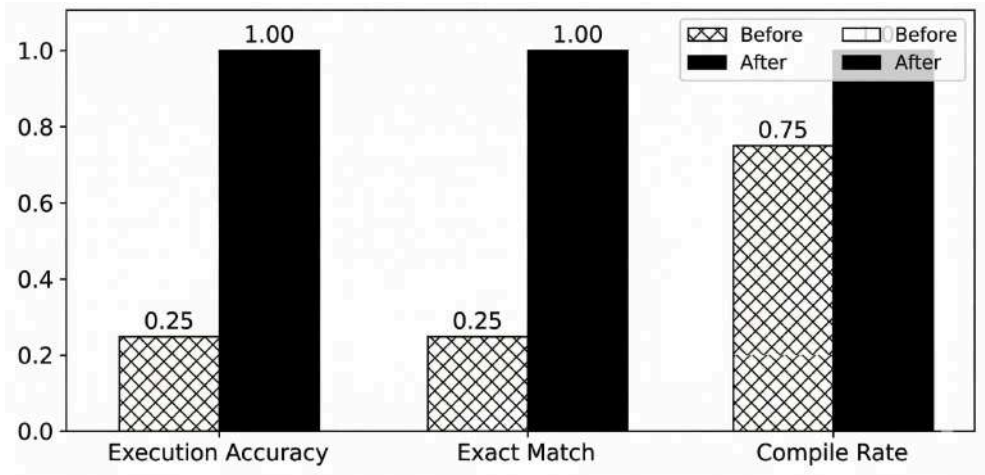
The difference between the two conditions was substantial. In the reduced “before” condition, the system achieved an **execution accuracy of 0.25**, an **exact match score of 0.25**, and a **compile rate of 0.75**. After restoring the full training set, all three measures increased to **1.00**. The latency also improved slightly, with the median generation time decreasing from **0.006362 seconds** to **0.004347 seconds**, and the median total latency decreasing from **0.018133 seconds** to **0.010294 seconds**.

These results show that the stored training examples are not only a supporting feature, but one of the main strengths of the current MVP. The before/after experiment demonstrates that the system can be improved through the training workflow without changing the source code. In particular, the examples help the system generate the correct join paths, grouping logic and calculation structure for more complex analytical questions. This result directly supports the product idea behind mxQueryChat: domain experts or analysts can improve future SQL generation by maintaining training examples through the application instead of modifying the implementation.

Table 6.9: Training Update Impact

Metric	Before	After	Interpretation
Execution Accuracy (ExecAcc)	0.25	1.00	Strong improvement after restoring relevant training examples
Exact Match (EM)	0.25	1.00	Strong improvement in structural SQL correctness
Compile Rate	0.75	1.00	Fewer invalid queries after restoring training examples
Generation latency median (s)	0.006362	0.004347	Slight improvement
Total latency median (s)	0.018133	0.010294	Improved end-to-end response time

Figure 11: Before/After Comparison of Training Impact



6.9 Usability observations

A limited usability signal was available from telemetry logs. The recorded events showed **10 successful question-to-run flows** and **4 positive feedback events**, while no negative feedback events were recorded. In total, the log contained **101 events**, including 36 generation events and 13 execution events.

Table 6.10: Telemetry-Based Usability Signals

Metric	Value
Successful question-to-run flows	10
Positive feedback events	4
Negative feedback events	0
Total logged events	101
Generation events	36
Execution events	13

Chapter 7 Discussion and Limitations

7.1 Purpose of this chapter

This chapter interprets the evaluation results and explains what they imply for mxQueryChat. The discussion focuses on what the measured results show about the current MVP, why the system performs well in some situations and poorly in others, and which limitations remain. In addition, the chapter outlines the most relevant directions for future work.

7.2 Interpretation of the main findings

The evaluation shows that mxQueryChat achieves the main goals of the implemented MVP. The system provides a complete local Text-to-SQL workflow: the user asks a question, the system generates SQL, the SQL is shown for review, execution is restricted to read-only queries, and the result is displayed in a table and, where suitable, as a chart. This corresponds to the intended product scope of a safe, self-hosted, domain-specific assistant for a local analytical DuckDB dataset. On the 20-question main domain test set, the system achieved perfect scores in execution accuracy, exact match and compile rate. This confirms that mxQueryChat works reliably for the supported business-style questions that are covered by the training examples and schema information. In this sense, the core MVP objective was achieved.

The held-out benchmark adds a stricter stress test to this result. It shows how the system behaves when questions are less directly supported by the stored examples. This test is useful because it identifies the boundary of the current fallback strategy, but it should not be confused with the main supported MVP scenario. On the local held-out benchmark set, execution accuracy and exact match were 0.00, while compile rate remained 0.90. This means that the system often produced SQL that was syntactically valid, but not yet semantically equivalent to the reference query. The result is important because it shows that the next quality improvement should focus on semantic verification and broader coverage of paraphrased questions.

The semantic error analysis gives more detail about these boundary cases. The paraphrase-based evaluation showed that difficult rewordings can still lead to wrong aggregation structures, wrong column choices or missing output dimensions. This does not reduce the value of the implemented MVP. Instead, it shows why the system correctly exposes generated SQL to the user and why the training-data workflow is important. The application is designed to support improvement over time, not to hide the generated query from the user.

Another important result is the strong effect of the training examples. In the before/after experiment, execution accuracy, exact match and compile rate increased from 0.25 / 0.25 / 0.75 to 1.00 / 1.00 / 1.00 after restoring the full training set. This is one of the clearest findings of the evaluation. It shows that the training-data workflow is not cosmetic, but directly improves the correctness of generated SQL. This result supports a central design decision of mxQueryChat. Instead of treating Text-to-SQL quality as a fixed property of the model, the system gives users a practical way to improve domain behavior by managing examples. For a company-specific assistant, this is valuable because business terminology and query patterns are often local to one dataset.

The safety results are a strong part of the system. All unsafe test cases were blocked, and no write operation reached the database. This confirms that the read-only guard and the additional execution limits behave consistently in the measured safety setting. For the intended use case of mxQueryChat, this is particularly important. Since the application is designed for analytical queries rather than database administration, it is acceptable to use conservative blocking if this reduces the risk of harmful SQL execution.

The latency results also support the practical usefulness of the MVP. In the main in-domain evaluation, generation and execution times were low enough to support interactive use. From the perspective of the user, the system behaves like a responsive assistant rather than a slow offline tool. This is important because a Text-to-SQL interface is only useful in practice if the answer is returned fast enough to support exploratory work.

Finally, the limited telemetry signals suggest that the main interaction flow is usable in practice, but this conclusion remains weak. The recorded events show successful question-to-run flows and positive feedback, but they do not replace a structured usability study with real participants. Therefore, these telemetry values can only be interpreted as preliminary implementation-level evidence.

7.3 Threats to validity

Several threats to validity must be considered when interpreting the evaluation.

First, internal validity may be affected by the dependence on the current model configuration, training examples, and vector store state. If the model, prompt behavior, or training data changes, the results may also change. This is particularly relevant because the before/after experiment shows that the training examples have a strong influence on performance. To reduce this risk, the evaluation used fixed question sets and recorded the environment details explicitly.

Second, construct validity is limited by the fact that different correctness metrics capture different aspects of system quality. Exact match is a strict metric and may penalize SQL that is different in structure but still semantically correct. Execution accuracy is more practical, but it depends on the quality of the reference SQL and the correctness of the reference result. In addition, the held-out benchmark results show that compile success does not guarantee semantic correctness. For this reason, the combination of quantitative metrics and qualitative semantic error analysis is more informative than any single metric alone.

Third, external validity is limited by the scope of the evaluation. The main dataset is synthetic, and the strongest measured results come from an in-domain scenario. Although this is appropriate for an MVP, it does not prove that the same system would perform equally well on larger, noisier, or less consistent real-world enterprise schemas. The local held-out benchmark provides a more difficult test than the main domain set, but it is still not the same as a large cross-domain benchmark.

Fourth, the usability evidence is limited. The telemetry logs provide useful signals, but they do not capture participant identity, task context, or detailed user reasoning. Therefore, they cannot replace a proper user study.

7.4 Limitations of the MVP

The first boundary of the MVP is its dependence on stored training examples. The measured before/after experiment shows that this dependence is also a strength of the design: when relevant examples are available, the system can produce correct SQL reliably. When examples are removed, correctness drops. This confirms that training-data management is a central mechanism of the MVP.

The second boundary is generalization beyond covered query patterns. The local held-out benchmark and the paraphrase-based test show that questions outside the strongest training coverage are harder for the current version. This is expected for a domain-specific MVP that prioritizes safety, transparency and editable training data over broad cross-domain benchmark performance. Future work can improve this area by adding better semantic verification, stronger retrieval and more systematic example coverage.

The third limitation is the handling of semantic ambiguity. The system currently attempts to generate SQL directly from the user question, even in cases where a clarification step would be more appropriate. As the semantic error analysis showed, the model may return executable SQL that still answers a different question than intended. A more robust system would detect ambiguity and ask follow-up questions before execution.

The fourth limitation concerns the safety layer. The current guardrails are strong enough to block unsafe

write operations, but they are still rule-based and conservative. This is useful for the MVP, but it may also block some safe edge cases. A more advanced solution would use deeper SQL parsing and policy enforcement instead of mainly relying on rule-based checks.

The fifth limitation is that training updates are synchronous. This keeps the implementation simple, but it can temporarily block the user interface while training is running. For the thesis scope, this trade-off is acceptable. However, for a more production-oriented system, asynchronous updates with progress feedback would be preferable.

The sixth limitation is database scope. The current system is built around DuckDB. Supporting additional database systems would require more robust dialect handling, schema extraction, and connection management.

7.5 Future work

Future work can build on the implemented MVP by extending the parts that already proved useful in the evaluation. A **first priority** is to expand the training-example coverage. The before/after experiment showed that training examples have a strong positive effect, so systematic example management is a practical way to increase supported query coverage.

A **second priority** is to add semantic verification. The current system already checks safety and compile validity. A future version could additionally check expected result dimensions, required grouping columns and common join paths before execution. A **third priority** is to improve clarification handling. If a question is underspecified, the system could ask a follow-up question instead of choosing one possible interpretation directly. This would support more reliable answers for ambiguous analytical questions.

A **fourth priority** is to improve the training workflow. Asynchronous training updates, better feedback on training progress, and stronger tooling for managing examples would make the system easier to maintain in practice. A **fifth priority** is to extend the system beyond DuckDB. Additional database connectors and better schema normalization would make the approach more useful in realistic enterprise settings.

Finally, future work should strengthen the evaluation itself. Useful next steps include larger held-out evaluations, more systematic semantic error analysis, regression testing after training changes and a structured usability study with real participants. This would make it clearer how well the system works outside the covered example set and how non-technical users judge the generated SQL in practice.

References

- [1] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang, “A Survey on Employing Large Language Models for Text-to-SQL Tasks,” *ACM Computing Surveys*, Vol. 58, No. 2, Article 54, 2025. URL: <https://arxiv.org/pdf/2407.15186>
- [2] Zijin Hong et al., “Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL,” *arXiv preprint arXiv:2406.08426*, pp. 1-20, 2024. URL: <https://arxiv.org/pdf/2406.08426>
- [3] Xiaohu Zhu et al., “Large Language Model Enhanced Text-to-SQL Generation: A Survey,” *arXiv preprint arXiv:2410.06011*, pp. 1-18, 2024. URL: <https://arxiv.org/pdf/2410.06011>
- [4] Tao Yu et al., “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911-3921, 2018. URL: <https://aclanthology.org/D18-1425.pdf>
- [5] Victor Zhong et al., “Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning,” *arXiv preprint arXiv:1709.00103*, pp. 1-12, 2017. URL: <https://arxiv.org/pdf/1709.00103>
- [6] Bailin Wang et al., “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers,” *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 7567-7578, 2020. URL: <https://aclanthology.org/2020.acl-main.677.pdf>
- [7] Mehdi Pourreza and Davood Rafiei, “DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction,” *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36, pp. 1-30, 2023. URL: <https://arxiv.org/pdf/2304.11015>
- [8] Rui Sun, Sercan O. Arik, Alexandre Muzio, Lesly Miculicich, Satya Kesav Gundabathula, Pengcheng Yin, Hanjun Dai, Hootan Nakhost, Rajarishi Sinha, Zifeng Wang, and Tomas Pfister, “SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL,” *Transactions on Machine Learning Research*, 2024. URL: <https://openreview.net/pdf?id=rloVZoKrX>
- [9] Jonathan Fürst, Catherine Kosten, Farhad Nooralahzadeh, Yi Zhang, and Kurt Stockinger, “Evaluating the Data Model Robustness of Text-to-SQL Systems Based on Real User Queries,” *Proceedings of EDBT 2025*, pp. 158–170, 2025. URL: <https://openproceedings.org/2024/conf/edbt/paper-61.pdf>
- [10] Qianhong Zhang et al., “EVOSCHEMA: TOWARDS TEXT-TO-SQL ROBUSTNESS AGAINST SCHEMA EVOLUTION,” *Proceedings of the VLDB Endowment (PVLDB)*, Vol. 18, pp. 3655-3667, 2025. URL: <https://www.vldb.org/pvldb/vol18/p3655-zhang.pdf>
- [11] Jinyang Li et al., “Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs,” *Advances in Neural Information Processing Systems*, Vol. 36, pp. 42330–42357, 2023. URL: <https://arxiv.org/pdf/2305.03111>

Appendix

The complete source code is provided on the accompanying CD. The appendix contains selected source code listings, while the full implementation is available in digital form.

Source Code Listing 1: AI Agent Setup (vannaagent.py)

```
# Combines ChromaDB vector store and Ollama LLM into a single Vanna-compatible agent.
class MXQueryVanna(ChromaDB_VectorStore, Ollama):
    def __init__(self, config=None):
        Ollama.__init__(self, config=config)
        ChromaDB_VectorStore.__init__(self, config=config)

def get_vanna():
    vn = MXQueryVanna(
        config={
            "model": os.getenv("OLLAMA_MODEL", "mistral"),
            "path": "vanna_chroma_store",
            "ollama_timeout": float(os.getenv("OLLAMA_TIMEOUT_SECONDS", "60")),
            "keep_alive": os.getenv("OLLAMA_KEEP_ALIVE", "30m"),
            "options": {
                "num_ctx": int(os.getenv("OLLAMA_NUM_CTX", "2048")),
                "num_predict": int(os.getenv("OLLAMA_NUM_PREDICT", "96")),
                "temperature": float(os.getenv("OLLAMA_TEMPERATURE", "0")), # deterministic
            },
        },
    )
    # Fallback handles API difference between Vanna versions.
    try:
        vn.connect_to_duckdb(path="mxquerychat.duckdb", read_only=True)
    except TypeError:
        vn.connect_to_duckdb(url="mxquerychat.duckdb", read_only=True)
    return vn
```

Source Code Listing 2: RAG Training (vannaagent.py)

```
# Schema facts loaded into ChromaDB to provide retrieval context during SQL generation.
BASE_TRAINING_DOCUMENTS = [
    "Main table: ticket_verkaeufe with columns monat, jahr, tarifverbund_id, ticket_code, umsatz_eur, anzahl, plz.",
    "To get state-level data: join ticket_verkaeufe.plz → postleitzahlen.plz → regionen_bundesland.bundesland_code2.",
    "To get tariff data: join ticket_verkaeufe.tarifverbund_id → tarifverbuende.tarifverbund_id.",
    "To get ticket product data: join ticket_verkaeufe.ticket_code → ticket_produkte.ticket_code.",
    "For state revenue: SUM(umsatz_eur) grouped by regionen_bundesland.bundesland_name.",
]
```

```

    "For tariff revenue: SUM(umsatz_eur) grouped by tarifverbuende.name.",
]

def train_from_examples(vn, examples_df):
    # Seed the vector store with schema knowledge before training on examples.
    for document in BASE_TRAINING_DOCUMENTS:
        vn.train(documentation=document)

    for _, row in examples_df.iterrows():
        question = row["question"].strip()
        sql = row["sql"].strip()
        description = row["description"].strip()

        if question and sql:
            vn.train(
                question=question,
                sql=sql,
                documentation=description if description else None,
            )

def get_exact_training_sql(question, examples_df):
    # Normalized match bypasses the LLM entirely for known questions.
    target = normalize_question(question)

    for _, row in examples_df.iterrows():
        stored_question = row["question"].strip()
        stored_sql = row["sql"].strip()
        if stored_sql and normalize_question(stored_question) == target:
            return stored_sql

    return None

```

Source Code Listing 3: Four-Tier Generation Pipeline (app.py)

```

if st.button("Send", type="primary"):
    question = st.session_state.question.strip()
    handled = False

    # Tier 0: reject off-topic or write-intent questions before any SQL work.
    local_block = get_local_guardrail_message(question)
    if local_block:
        st.warning(local_block)
        handled = True

    # Tier 1: in-memory session cache -- instant reuse within the same session.
    if not handled:

```

```

cached_sql = st.session_state.sql_cache.get(question.lower())
if cached_sql:
    st.session_state.generated_sql = cached_sql
    st.session_state.generation_notes = ["Loaded from session cache."]
    st.success("Used cached SQL.")
    handled = True

# Tier 2: exact match in training CSV -- no LLM call required.
if not handled:
    examples_df = load_training_examples()
    direct_sql = get_exact_training_sql(question, examples_df)
    if direct_sql:
        st.session_state.generated_sql = direct_sql
        st.session_state.sql_cache[question.lower()] = direct_sql
        st.session_state.generation_notes = ["Matched from training examples."]
        st.success("Used training example SQL.")
        handled = True

# Tier 3: deterministic template planner -- keyword-based, no LLM required.
if not handled:
    template_sql, template_note = build_template_sql(question)
    if template_sql and not validate_sql_compiles(template_sql):
        st.session_state.generated_sql = template_sql
        st.session_state.sql_cache[question.lower()] = template_sql
        st.session_state.generation_notes = [template_note]
        st.success("Used template planner.")
        handled = True

# Tier 4: LLM fallback with up to three retry attempts.
if not handled:
    with st.spinner("Generating SQL..."):
        vn = get_vanna_cached()
        sql, notes, error = generate_sql_with_retry(
            generate_sql_fn=lambda prompt: vn.generate_sql(prompt),
            question_text=question,
            schema_tree=get_schema_tree(),
            compile_sql_fn=validate_sql_compiles,
            timeout_seconds=65,
            run_with_timeout_fn=run_with_timeout,
        )
    if error:
        st.error("Could not generate SQL for this question.")
    else:
        st.session_state.generated_sql = sql
        st.session_state.sql_cache[question.lower()] = sql

```

Source Code Listing 4: LLM Retry Logic (src/core/query_logic.py)

```

def build_first_pass_prompt(question, schema_tree):
    relevant_tables = guess_relevant_tables(question, schema_tree)
    table_lines = []
    for table in relevant_tables:
        columns = ", ".join(col for col, _ in schema_tree[table][:20])
        table_lines.append(f"- {table}({columns})")

    return (
        "Generate DuckDB SQL for mxquerychat.\n"
        "Use ONLY the tables and columns listed below.\n"
        "Return ONLY SQL, no explanation.\n\n"
        f"Question:\n{question}\n\n"
        "Available tables and columns:\n"
        + "\n".join(table_lines)
    )

def generate_sql_with_retry(
    generate_sql_fn, question_text, schema_tree,
    compile_sql_fn, timeout_seconds, run_with_timeout_fn
):
    notes = []

    def run_attempt(prompt_text):
        raw_sql, error = run_with_timeout_fn(
            lambda: generate_sql_fn(prompt_text), timeout_seconds
        )
        if error:
            status = "timeout" if "timeout" in str(error).lower() else "model_error"
            return "", status, str(error)

        # NO_MATCH signals the model explicitly cannot answer from this schema.
        if (raw_sql or "").strip().upper() == "NO_MATCH":
            return "", "no_match", "NO_MATCH"

        # Strip markdown fences or surrounding explanation text.
        sql = extract_sql_only(raw_sql or "")
        if not sql:
            return "", "non_sql", ""

        # Validate against DuckDB schema using EXPLAIN (dry run, no execution).
        compile_error = compile_sql_fn(sql)
        if compile_error:
            return sql, "compile_error", compile_error

        return sql, "ok", ""

    # Attempt 1: schema-guided prompt.
    sql_1, status_1, detail_1 = run_attempt(

```

```

        build_first_pass_prompt(question_text, schema_tree)
    )
    if status_1 == "ok":
        return sql_1, notes, ""
    if status_1 in ("timeout", "model_error"):
        return "", notes, status_1

    # Attempt 2: retry with previous SQL and DuckDB error included.
    sql_2, status_2, detail_2 = run_attempt(
        build_retry_prompt(question_text, sql_1, detail_1, schema_tree)
    )
    if status_2 == "ok":
        return sql_2, notes, ""
    if status_2 in ("timeout", "model_error"):
        return "", notes, status_2

    # Attempt 3: strict final prompt -- forces SQL or explicit NO_MATCH token.
    sql_3, status_3, _ = run_attempt(
        build_final_retry_prompt(question_text, sql_2 or sql_1, detail_2 or detail_1,
        schema_tree)
    )
    if status_3 == "ok":
        return sql_3, notes, ""

    return "", notes, "no_match"

```

Source Code Listing 5: SQL Safety Check (sql_guard.py)

```

BLOCKED_KEYWORDS = [
    "insert", "update", "delete", "merge", "replace",
    "create", "alter", "drop", "truncate",
    "grant", "revoke", "attach", "detach",
    "copy", "export", "import", "call", "execute",
    "pragma", "recursive",
]

def validate_read_only_sql(sql):
    if not sql or not sql.strip():
        return False, "SQL is empty."

    cleaned = sql.strip()

    # Multiple semicolons indicate a multi-statement injection attempt.
    if cleaned.count(";") > 1:
        return False, "Blocked: multiple SQL statements are not allowed."

    if ";" in cleaned and not cleaned.rstrip().endswith(";"):

```

```

        return False, "Blocked: semicolon must only appear at the end."

cleaned = cleaned.rstrip(";").strip()

# Only SELECT and WITH are permitted entry points for read-only queries.
first_word = cleaned.split()[0].lower()
if first_word not in ("select", "with"):
    return False, "Blocked: only SELECT/WITH queries are allowed."

for keyword in BLOCKED_KEYWORDS:
    # Word boundary prevents false positives (e.g. "dropdown" matching "drop").
    if re.search(rf"\b{keyword}\b", cleaned.lower()):
        return False, f"Blocked: keyword '{keyword.upper()}' is not allowed."

return True, "OK"

```

Source Code Listing 6: Safe Query Execution (src/db/execution_policy.py)

```

@dataclass(frozen=True)
class ExecutionPolicy:
    max_rows: int = 1000
    timeout_seconds: int = 15
    max_joins: int = 6
    max_ctes: int = 4
    max_sql_chars: int = 20000

def validate_sql_complexity(sql, policy):
    if len(sql) > policy.max_sql_chars:
        return False, f"Query too long (limit: {policy.max_sql_chars} characters)."

    join_count = len(re.findall(r"\bjoin\b", sql, re.IGNORECASE))
    if join_count > policy.max_joins:
        return False, f"Too many JOINS ({join_count}, limit: {policy.max_joins})."

    cte_count = len(re.findall(r"\b\w+\s+as\s*\(", sql, re.IGNORECASE))
    if cte_count > policy.max_ctes:
        return False, f"Too many subqueries ({cte_count}, limit: {policy.max_ctes})."

    return True, "OK"

def run_query_with_timeout(duckdb_path, sql, timeout_seconds):
    # Wrap query with a hard row cap regardless of what the original query requests.
    sql_with_limit = (
        "SELECT * FROM (\n"
        f"{sql}\n"
        ") AS result\n"
    )

```

```

        "LIMIT 1000"
    )

    # Run in a separate subprocess to isolate crashes and enforce the timeout.
    ctx = mp.get_context("spawn")
    queue = ctx.Queue()
    process = ctx.Process(target=_query_worker, args=(duckdb_path, sql_with_limit, queue))
    process.start()
    process.join(timeout_seconds)

    if process.is_alive():
        # Forcefully terminate if the query has not finished within the deadline.
        process.terminate()
        process.join()
        return pd.DataFrame(), 0.0, f"Query timed out after {timeout_seconds} seconds."

    try:
        payload = queue.get(timeout=2)
    except Empty:
        return pd.DataFrame(), 0.0, "No result returned by worker."

    if not payload.get("ok"):
        return pd.DataFrame(), 0.0, payload.get("error", "Unknown error.")

    return payload["df"], payload["elapsed"], ""

```

Source Code Listing 7: Deterministic Template Planner (src/core/query_logic.py)

```

def build_template_sql(question):
    q = question.lower()

    # Detect topic flags from question keywords (bilingual: English and German).
    wants_revenue = any(word in q for word in ["revenue", "umsatz", "sales"])
    wants_state = any(word in q for word in ["state", "bundesland", "region"])
    wants_month = any(word in q for word in ["month", "monat", "monthly"])
    wants_tariff = any(word in q for word in ["tariff", "tarifverbund", "association"])
    wants_ticket = any(word in q for word in ["ticket type", "ticketart", "ticket product"])
    wants_total = any(word in q for word in ["total", "gesamt", "overall"])

    # Most-specific combinations are checked first to avoid partial matches.
    if wants_revenue and wants_state and wants_tariff and wants_month:
        return ""

    SELECT tv.jahr, tv.monat, t.name AS tariff, rb.bundesland_name, SUM(tv.umsatz_eur) AS revenue
    FROM ticket_verkaeuft tv
    JOIN tarifverbund t ON tv.tarifverbund_id = t.tarifverbund_id
    JOIN postleitzahlen p ON CAST(tv.plz AS VARCHAR) = p.plz
    JOIN regionen_bundesland rb ON p.bundesland_code2 = rb.bundesland_code2
    GROUP BY tv.jahr, tv.monat, t.name, rb.bundesland_name

```

```

ORDER BY tv.jahr, tv.monat
        """', "Template: revenue by tariff, state, and month."

    if wants_revenue and wants_state:
        return ""
SELECT tv.jahr, rb.bundesland_name, SUM(tv.umsatz_eur) AS revenue
FROM ticket_verkaeufer tv
JOIN postleitzahlen p      ON CAST(tv.plz AS VARCHAR) = p.plz
JOIN regionen_bundesland rb ON p.bundesland_code2 = rb.bundesland_code2
GROUP BY tv.jahr, rb.bundesland_name
ORDER BY revenue DESC
        """', "Template: revenue by state."

    if wants_revenue and wants_tariff:
        return ""
SELECT tv.jahr, t.name AS tariff, SUM(tv.umsatz_eur) AS revenue
FROM ticket_verkaeufer tv
JOIN tarifverbuende t ON tv.tarifverbund_id = t.tarifverbund_id
GROUP BY tv.jahr, t.name
ORDER BY revenue DESC
        """', "Template: revenue by tariff association."

    if wants_revenue and wants_month:
        return ""
SELECT tv.jahr, tv.monat, SUM(tv.umsatz_eur) AS revenue
FROM ticket_verkaeufer tv
GROUP BY tv.jahr, tv.monat
ORDER BY tv.jahr, tv.monat
        """', "Template: revenue by month."

    if wants_revenue and wants_ticket:
        return ""
SELECT tv.jahr, tp.ticket_name, SUM(tv.umsatz_eur) AS revenue
FROM ticket_verkaeufer tv
JOIN ticket_produkte tp ON tv.ticket_code = tp.ticket_code
GROUP BY tv.jahr, tp.ticket_name
ORDER BY revenue DESC
        """', "Template: revenue by ticket type."

    if wants_revenue and wants_total:
        return ""
SELECT SUM(tv.umsatz_eur) AS total_revenue
FROM ticket_verkaeufer tv
        """', "Template: total revenue."

# No template matched -- caller falls through to LLM generation.
return "", ""

```

Declaration

I declare that this Bachelor Thesis has been completed by myself independently without outside help and only the defined sources and study aids were used.

Hamburg, _____